

Traffic-MoE: A Sparse Foundation Model for Network Traffic Security Analysis

Jiajun Zhou, Changhui Sun, Wentao Fu, Meng Shen, *Member, IEEE*, Shanjing Yu, and Qi Xuan, *Senior Member, IEEE*

Abstract—As adversaries increasingly weaponize encryption and protocol obfuscation to evade traffic detection, traditional methods are rendered obsolete, necessitating deep learning to unmask sophisticated threats. However, the prohibitive computational costs of existing large models create a critical defense gap, hindering their deployment in real-time and throughput-sensitive environments. To close this vulnerability, we introduce *Traffic-MoE*, a sparse foundation model tailored for traffic security analysis. By dynamically routing traffic tokens to a small subset of specialized experts, *Traffic-MoE* effectively decouples model capacity from computational overhead. Extensive evaluations across four security-oriented tasks demonstrate that *Traffic-MoE* achieves state-of-the-art or highly competitive performance compared to leading competitors. Crucially, it delivers a 70.42% increase in throughput, reduces inference latency by 41.39% while significantly optimizing GPU memory consumption. Beyond efficiency, *Traffic-MoE* exhibits superior robustness against adversarial traffic shaping and maintains strong detection capabilities in few-shot scenarios, establishing a scalable and resilient paradigm for modern network traffic security analysis.

Index Terms—Network Traffic Analysis, Mixture-of-Experts, Pretraining

I. INTRODUCTION

Network communication has rapidly evolved with the widespread adoption of encryption, heterogeneous device ecosystems, and emerging paradigms such as IoT and Web3 [1, 2]. Today, TLS-based services, device-to-cloud telemetry, and decentralized protocols dominate Internet traffic, obscuring application semantics from traditional packet inspection [3]. While these advancements improve privacy and availability, they also pose a critical security challenge: malicious activities, such as botnet communications and credential misuse, can hide within encrypted traffic and remain indistinguishable from benign services at the packet level [4, 5].

Traffic analysis is fundamental to network defense, supporting tasks such as intrusion detection [6], anomaly detection

[7] and service attribution [8]. However, classical machine-learning approaches rely on handcrafted statistical features or protocol signatures, which are fragile under encryption, obfuscation, and protocol mimicry. To address this, recent work leverages deep learning models that operate directly on raw byte sequences or image-based representations, achieving higher accuracy on heterogeneous and encrypted traffic [9, 10]. Following trends in computer vision and natural language processing, state-of-the-art (SOTA) pre-trained traffic analysis models increasingly leverage large transformer-based architectures to further improve downstream performance [11–13].

Despite these advances, most models employ *dense* architectures that activate all parameters during inference, causing computational cost to scale with model size. As model capacity grows, inference latency becomes prohibitively high, inevitably inducing detection blind spots and creating a critical bottleneck for real-time network traffic security analysis. Although pre-trained models capture rich protocol semantics, their dense architectures force a trade-off between detection depth and system availability: large models may perform well on offline benchmarks yet struggle in operational environments where high throughput and low latency are critical. A practical defense system therefore requires models that preserve semantic expressiveness while enabling efficient inference.

We address this security-efficiency gap by rethinking how large-scale traffic analysis models should allocate computation. We observe that network traffic behaviors are structurally heterogeneous rather than uniformly distributed, as malware beacons, VPN tunnels, Web3 relay traffic, and benign application flows all exhibit distinct byte-level patterns and temporal dynamics [5]. A uniform dense architecture inefficiently forces the entire network to process all patterns, whereas a specialized-expert model can dynamically activate only the parameter subspaces most relevant to the traffic domain being processed [14]. This leads to our central insight: *Efficiency in network traffic modeling does not necessarily stem from shrinking models, but rather from selectively activating model capacity during inference* [15].

In this work, we propose *Traffic-MoE*, a sparse Mixture-of-Experts (MoE) [14] foundation model for network traffic analysis. Instead of activating all parameters during inference, *Traffic-MoE* routes each token through a small subset of specialized experts, decoupling total detection capability from computational overhead. This design enables scaling the model parameter count by orders of magnitude without proportionally increasing inference cost, making large-scale pre-trained traffic analysis models feasible for latency-sensitive scenarios. To

This work was supported in part by National Natural Science Foundation of China (No. 62503423), in part by the National Key Research and Development Program of China (No. 2025YFA1510900), in part by the Key Research and Development Program of Zhejiang Province (No. 2026C02A1233), in part by the Yangtze River Delta Science and Technology Innovation Community Joint Research Project (No. 2026ZY03003, No. 2025CSJGG01000). (Corresponding author: Jiajun Zhou.)

Jiajun Zhou, Shanjing Yu, Qi Xuan, Changhui Sun, Wentao Fu are with the Institute of Cyberspace Security, Zhejiang University of Technology, Hangzhou 310023, China, with the Binjiang Cyberspace Security Institute of ZJUT, Hangzhou, 310056, China, and with the Soovar Technologies Co., Ltd., Hangzhou 310056, China (e-mail: jjzhou@zjut.edu.cn).

Meng Shen is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: shenmeng@bit.edu.cn).

support heterogeneous network flows, we further develop *Traffic2Token*, a unified byte-level representation method combining protocol metadata with selective payload segments, facilitating the unified modeling of cross-layer traffic patterns. *Traffic-MoE* is trained using autoregressive self-supervision over massive unlabeled traffic corpora, capturing protocol semantics, flow directionality, and temporal dependencies. During downstream adaptation, *Traffic-MoE* can be fine-tuned for intrusion detection, service classification, VPN/Tor analysis, and other security tasks while preserving real-time inference performance, making it suitable not only for benchmarking but for operational deployment. The main contributions of this work are summarized as follows:

- **A sparse foundation model for network traffic security analysis.** We propose *Traffic-MoE*, the first MoE architecture tailored to traffic modeling, enabling large model capacity while maintaining real-time inference, bridging the efficiency gap inherent in dense pre-trained models.
- **A unified learning framework enabling expert specialization across traffic semantics.** We support heterogeneous network flows through *Traffic2Token* and balanced sparse routing, allowing experts to specialize in distinct traffic behaviors and improving security-oriented generalization.
- **Comprehensive evaluations demonstrate the superiority and efficiency.** Extensive experiments validate that *Traffic-MoE* outperforms SOTA competitors, achieving a 70.42% throughput increase and a 41.39% latency reduction while optimizing memory utilization. It also exhibits robust performance in few-shot scenarios and under distribution shifts.

II. RELATED WORK

A. Statistical Machine Learning Methods

Early traffic analysis methods rely on Deep Packet Inspection (DPI), matching predefined signatures within packet headers or payloads [16]. While effective in unencrypted scenarios, DPI incurs high computational overhead and becomes unreliable as modern protocols increasingly employ end-to-end encryption and payload obfuscation.

To overcome payload visibility reliance, statistical feature-based machine learning (ML) approaches extract handcrafted features for classification. Moore et al. [17] reduce dependence on port-based heuristics by focusing on payload features, while Saber et al. [18] combine PCA with SVM to model time-based statistical patterns. AppScanner [19] trains a Random Forest on 54 statistical features to identify mobile applications. FlowPrint [20] leverages clustering to capture temporal correlations among network flows and generates application-specific traffic fingerprints, enabling the identification of previously unseen applications. Although computationally efficient, these methods depend heavily on feature engineering and struggle to capture deep semantic patterns, rendering them inadequate against encrypted or adversarial traffic conditions.

B. Deep Learning Methods

Following the rise of deep learning (DL), end-to-end models have been widely adopted to automatically extract features

from raw traffic data, reducing reliance on handcrafted features. These approaches handle traffic using various data representations and model architectures. For sequence modeling, FS-Net [10] employs a bidirectional GRU-based encoder-decoder architecture to directly learn features from encrypted traffic flows. For image-based representations, ATVITSC [21] transforms payload bytes into images and combines vision transformers with LSTM modules to jointly capture spatial and temporal dependencies. Graph-based models further treat flows as structured relational data. TFE-GNN [22] encodes packet headers and payloads as graph nodes, while Zhou et al. [23] introduce hypergraph structure that capture higher-order correlations through KNN-derived hyperedges. These methods demonstrate the ability of deep neural networks to learn semantic structure beyond handcrafted features. However, they remain heavily dependent on large-scale labeled datasets, which are expensive to construct in real-world environments [24]. Label scarcity is especially pronounced in critical long-tail classes such as zero-day attacks and malware variants, where samples are sparse and annotation requires expert knowledge. As a result, even advanced DL approaches experience significantly degraded performance when deployed in practical scenarios with limited labeled data.

C. Large Pre-trained Methods

Recent studies introduce pre-training paradigm into traffic analysis. Models such as ET-BERT [11], NetGPT [12], and TrafficFormer [13] perform self-supervised learning on large-scale traffic corpora to acquire generalizable representations. To address the heavy computational overhead of standard Transformers, recent efforts like YaTC [25] and NetMamba [26] have explored efficiency optimizations, with the latter leveraging Mamba for faster inference. These methods significantly improve performance on network traffic detection under limited supervision, demonstrating the promise of the pre-train and fine-tune paradigm. However, these models generally remain dense, where all parameters are activated during inference. As noted in [14], model capacity and inference cost remain tightly coupled, leading to resource bottlenecks that limit deployment in real-time operational networks.

III. THREAT MODEL

Deployment Scenario. Figure 1 illustrates a hierarchical monitoring architecture that deploys a two-stage filter at the network edge to balance throughput and inspection depth. A primary lightweight filter first intercepts plain threats [27], while suspicious or encrypted flows are mirrored for deep out-of-path inspection. This design ensures minimal latency overhead and triggers real-time policy updates (e.g., blocking or QoS adjustment) [28]. Given massive backbone traffic, the analysis engine must maximize parameter efficiency to sustain low-latency processing without detection blind spots.

Adversary Capabilities. We assume a sophisticated adversary aiming to disrupt services or exfiltrate data while evading detection. The adversary employs encryption and anonymization to conceal payload contents and communication endpoints [29]. To defeat statistical analysis, adversaries

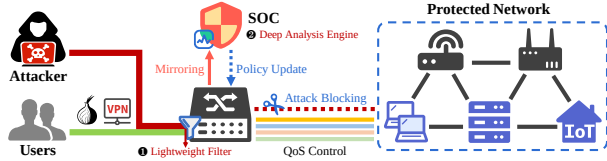


Fig. 1: Threat model and operational scenario.

employ traffic shaping techniques to distort deterministic flow fingerprints [30, 31]. Furthermore, they actively exploit distribution shift strategies to launch zero-day attacks or generate out-of-distribution (OOD) traffic, attempting to bypass models trained on stationary datasets.

Knowledge and Constraints. Under a black-box assumption, adversaries are cognizant of the presence of traffic analysis systems yet remain agnostic to the specific defense architecture, lacking white-box access to model parameters or gradients [32]. Furthermore, attack strategies are subject to strict functional constraints, e.g., traffic manipulation must neither compromise the reliability of the command-and-control (C2) channel nor render communication indistinguishable from random noise. This is because perfect mimicry typically incurs unacceptable operational costs [24, 33].

Defender Capabilities. While the defender possesses full visibility of mirrored traffic headers and encrypted payloads, operations are conducted under strict constraints. Specifically, due to privacy regulations and protocols implementing Perfect Forward Secrecy (PFS), payloads cannot be decrypted, compelling the analysis engine to rely exclusively on plaintext metadata, temporal sequences, and underlying behavioral patterns [34, 35]. Furthermore, we assume a trusted infrastructure with a secure traffic collection and analysis pipeline, rendering threats such as physical compromise of the SOC or pre-training data poisoning out of scope.

IV. TRAFFIC-MoE ARCHITECTURE

Network traffic exhibits heterogeneous behavioral patterns that necessitate domain-specific modeling rather than uniform dense computation. Meanwhile, real-world security monitoring systems require low-latency processing to prevent packet drops, making dense large-scale models impractical. These challenges create a deployment gap: semantic modeling of network traffic is increasingly effective but not computationally feasible in operational environments. To address these constraints, *Traffic-MoE* is designed as a sparse foundation model for network traffic analysis that balances semantic expressiveness with computational efficiency. Instead of shrinking model capacity, *Traffic-MoE* activates only a subset of model parameters per input, enabling scalability without proportional inference overhead. This sparse activation strategy adapts the computational pathway to specific traffic semantics, enabling domain specialization across heterogeneous flows.

Figure 2 presents the overall workflow. Specifically, *Traffic-MoE* consists of four components: ❶ Traffic2Token converts raw flows into structured byte-level token sequences that preserve critical metadata and structural patterns; ❷ A hierarchical transformer backbone models temporal dependencies

across packets to obtain contextualized embeddings; ❸ Sparse MoE layers replace dense feed-forward layers and activate only top- k experts based on token semantics, combining shared experts with domain-specialized ones; ❹ A pretrain-finetune pipeline enables large-scale self-supervised learning and downstream task adaptation, supporting scenarios such as service classification and malicious traffic detection. Together, these components form a unified architecture that preserves semantic modeling capability while ensuring high efficiency in throughput-sensitive environments.

A. Traffic Tokenization

Raw traffic collected from operational networks contains heterogeneous communication patterns across applications, protocols, and devices. Directly modeling raw packets is ineffective as full headers introduce substantial noise while encrypted payloads limit semantic visibility. To bridge this gap, we design a Traffic2Token module that converts raw traffic into structured token sequences, preserving byte-level behavioral signals without protocol-specific assumptions.

1) *Packet-Based Flow Serialization:* We first reconstruct session-level flows from packet traces using five-tuple identifiers (IPs, ports, and protocol) establishing the basic processing unit for Traffic2Token. To minimize noise from randomized or transport-irrelevant header fields, each packet is transformed into a compact and stable representation. Specifically, we extract six packet-level attributes reflecting transport and timing dynamics: packet length, direction, TCP flags, inter-arrival time, protocol type, and payload length, as shown in Figure 3. These attributes are serialized into a fixed-length byte sequence. In parallel, we sample the first J payload bytes to capture representative structure patterns while controlling sequence length. The metadata bytes and sampled payload bytes are then concatenated to form the packet-level hex sequence. For each flow, the hex representations of the first K packets are chronologically concatenated to construct a flow-level hex sequence. This packet-based serialization preserves communication directionality, burstiness, and timing structure.

2) *Tokenization and Embedding:* Network protocols often exhibit short-range structural regularities such as protocol-specific header fields, framing patterns, and data boundaries. To expose such correlations, we apply a sliding window to the flow hex sequence to generate byte bigrams. This transformation yields a fine-grained sequence highlighting local structural cues without full header retention. We then treat the bigram sequences from all flows as a corpus and employ the WordPiece [36] algorithm to construct a sub-byte-level vocabulary $\mathcal{V}$. WordPiece adaptively identifies statistically significant bigram combinations and assigns them as token units. This data-driven vocabulary captures recurrent traffic patterns while avoiding vocabulary explosion. Furthermore, to enhance the model’s perception of the session flow’s hierarchical structure, we introduce function-specific markers into the sequence: [PD] for the beginning of each packet sequence, [PY] separating metadata fields from payload bytes, [PAD] for sequence padding, [END] denoting flow termination, and [UNK] representing low-frequency out-of-vocabulary byte patterns. These structural markers ensure that packet boundaries remain clearly

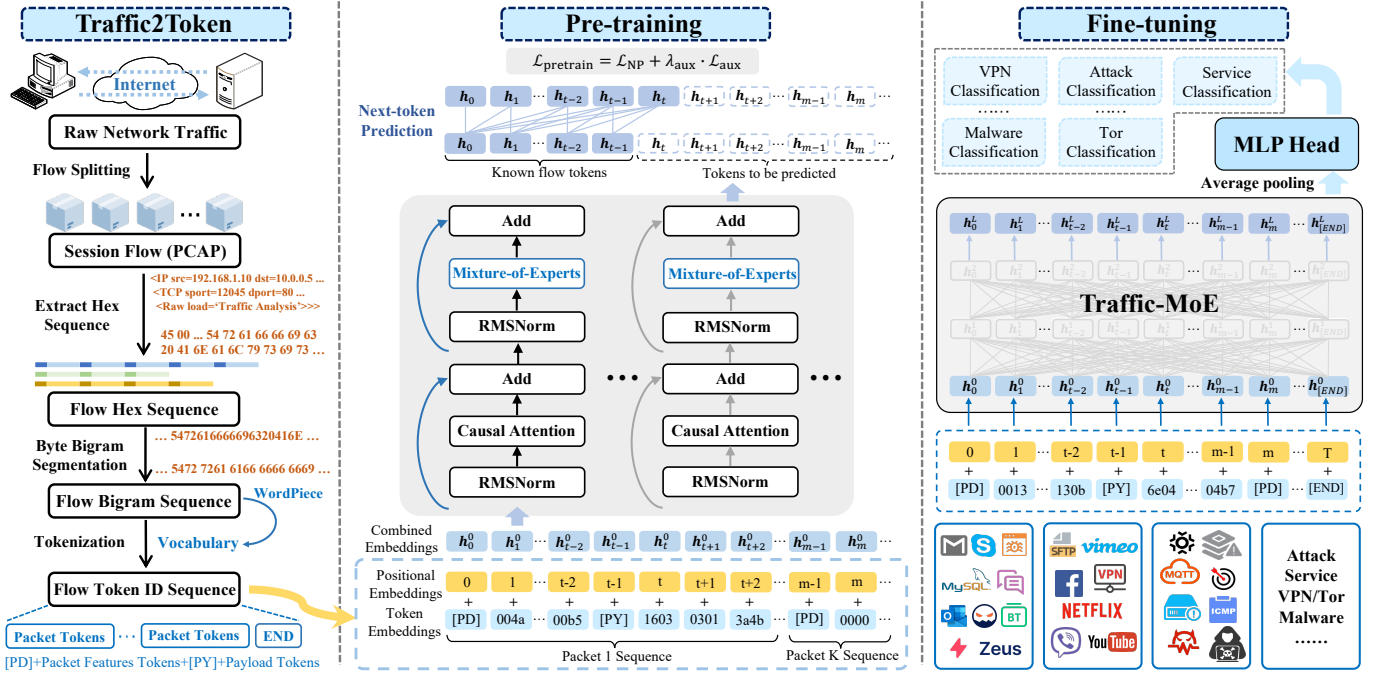


Fig. 2: The overall framework of *Traffic-MoE*. It consists of the Traffic2Token serialization module, the sparse Mixture-of-Experts (MoE) backbone, and the pre-training/fine-tuning pipeline for downstream security tasks.

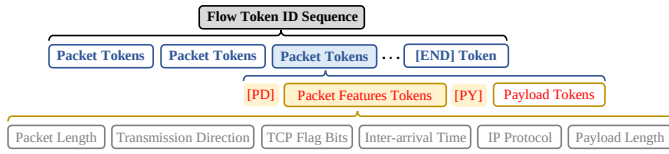


Fig. 3: Illustration of flow token sequence representation.

identifiable to the model when processing long sequences, preventing the semantic collapse of flow information within a flattened byte stream. Ultimately, the bigram tokens and special markers are mapped to Token IDs via the learned vocabulary. Each Token ID is embedded into a dense vector and summed with a positional encoding, producing the flow token sequence representation: $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$, where $\mathbf{x}_t \in \mathbb{R}^d$ is the initial token embedding in position t , T is the sequence length, and d is the embedding dimension. This unified embedding supports both local byte-level pattern modeling and global packet-level temporal reasoning, enabling robust pre-training and sparse expert routing.

B. Backbone with Sparse Expert Layers

Figure 4 illustrates the backbone design of *Traffic-MoE*, which interleaves causal masked self-attention with sparse MoE layers to address the dual nature of network traffic. This architecture is designed to capture two distinct dimensions of traffic semantics: universal protocol syntax and heterogeneous application behaviors. In the sequence dimension, dense self-attention operates globally across tokens to model the shared state transitions and temporal dependencies inherent in the TCP/IP protocol suite. This builds a robust contextual foundation of the underlying “protocol grammar” (e.g., handshake).

In the token dimension, sparse MoE layers independently process individual tokens via dynamic routing to decouple and extract high-order, domain-specific semantic features. By stacking these layers, the backbone achieves a progressive, layer-wise abstraction, transitioning from global syntax aggregation to localized feature purification. This collaborative design, supplemented by shared expert anchors, ensures that *Traffic-MoE* can achieve deep specialization for complex downstream tasks without losing its grasp on fundamental protocol rules, providing a comprehensive and hierarchical representation of network flows.

1) *Causal Masked Self-Attention*: Network flows evolve through ordered protocol phases such as handshake exchanges, tunneling encapsulation, retransmissions, and periodic keep-alives, which manifest as temporal structures in their byte-level token sequences. To model these sequential dependencies while respecting real-world monitoring constraints where future packets are unavailable at inference time, *Traffic-MoE* employs a causal masked multi-head self-attention mechanism.

Given hidden states $\mathbf{H}^{(l-1)} \in \mathbb{R}^{T \times d}$ from the $(l-1)$ -th block ($\mathbf{H}^0 = \mathbf{X}$), we first apply Root Mean Square Layer Normalization (RMSNorm) [37] to stabilize feature scales across heterogeneous flow segments and improve training stability. Subsequently, to encode temporal offsets between packets, such as burst intervals or phase transitions, we incorporate Rotary Positional Embeddings (RoPE) [38] into the query and key projections. Unlike absolute positional encodings, RoPE mathematically injects relative position information by rotating the feature vectors in a shared frequency space. The query ($\mathbf{Q}$), key ($\mathbf{K}$), and value ($\mathbf{V}$) matrices for the j -th attention head

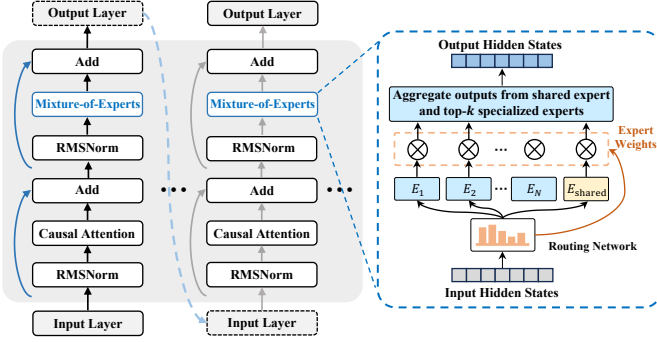


Fig. 4: Illustration of *Traffic-MoE* backbone architecture.

are computed as:

$$\begin{aligned} Q_j^{(l)} &= \text{RoPE} \left(\text{RMSNorm} \left(H^{(l-1)} \right) \cdot W_{Q,j}^{(l)} \right), \\ K_j^{(l)} &= \text{RoPE} \left(\text{RMSNorm} \left(H^{(l-1)} \right) \cdot W_{K,j}^{(l)} \right), \\ V_j^{(l)} &= \text{RMSNorm} \left(H^{(l-1)} \right) \cdot W_{V,j}^{(l)}, \end{aligned} \quad (1)$$

where $W_{\{Q,K,V\},j}^{(l)} \in \mathbb{R}^{d \times d_m}$ are learnable projection matrices. This formulation enables the attention mechanism to capture relative sequential distance between flow tokens, identifying traffic patterns invariant to absolute sequence positions (e.g., attack signatures appearing at varying offsets).

To ensure traffic causality where future packets cannot influence current state, we apply a strictly lower-triangular causal mask $M \in \mathbb{R}^{T \times T}$. The attention output is:

$$\begin{aligned} O_j^{(l)} &= \text{Softmax} \left(\frac{Q_j^{(l)} (K_j^{(l)})^\top}{\sqrt{d_m}} + M \right) \cdot V_j^{(l)} \\ M_{tp} &= \begin{cases} 0, & p \leq t \\ -\infty, & p > t \end{cases} \end{aligned} \quad (2)$$

This causal mask ensures that the token at position t can only attend to information from itself and preceding positions. Finally, the outputs from all m attention heads are concatenated and projected to form the residual update:

$$\tilde{H}^{(l)} = H^{(l-1)} + \left[O_1^{(l)} \parallel O_2^{(l)} \parallel \dots \parallel O_m^{(l)} \right] \cdot W_O^{(l)}, \quad (3)$$

where $W_O^{(l)} \in \mathbb{R}^{d \times d}$ is the output projection matrix, and $\tilde{H}^{(l)}$ is the intermediate hidden state. This design enables autoregressive learning of traffic semantics, effectively modeling the state transition logic of network protocols.

2) *Mixture-of-Experts Layer with Sparse Routing*: Following the causal masked self-attention module, the intermediate hidden state $\tilde{H}^{(l)}$ is processed by a sparse MoE layer. Standard Transformers typically employ a dense Feed-Forward Network (FFN) that applies identical parameters to all tokens. However, network traffic combines universal protocol regularities (e.g., transport headers, handshake patterns) with highly heterogeneous behaviors (e.g., application-specific traffic fingerprints). To accommodate this heterogeneity without incurring a proportional increase in computational cost, *Traffic-MoE* replaces

the dense FFN with a hybrid MoE layer composed of a shared expert and multiple specialized experts.

To decouple general protocol knowledge from specific traffic behaviors, we design a dual-pathway computation. First, the input $\tilde{H}^{(l)}$ undergoes normalization via RMSNorm to produce $Z^{(l)} = \text{RMSNorm}(\tilde{H}^{(l)})$. This normalized representation is then routed to two distinct expert groups, as defined below:

Shared Expert for Universal Protocol Semantics. To capture invariant traffic structures distinct from application-specific payloads, we employ a persistently activated *shared expert*, E_{shared} . Unlike the competitive routing of specialized experts, the shared expert utilizes an independent gate to adaptively regulate the injection of general traffic knowledge:

$$O_{\text{shared}}^{(l)} = \text{Sigmoid} \left(Z^{(l)} w_\Delta \right) \odot E_{\text{shared}} \left(Z^{(l)} \right), \quad (4)$$

where $w_\Delta \in \mathbb{R}^d$ is a projection vector, and $\odot$ denotes element-wise multiplication broadcasted over the feature dimension.

Specialized Experts for Heterogeneous Patterns. Simultaneously, N specialized experts $\{E_e\}_{e=1}^N$ are established to handle diverse traffic patterns. A routing network computes a probability distribution over these experts, selecting the Top- k relevant experts to process each token. The routing scores $S^{(l)}$ and the sparse routing weights $\tilde{S}^{(l)}$ are computed as:

$$S^{(l)} = \text{Softmax} \left(Z^{(l)} W_* \right), \quad \tilde{S}_{ue}^{(l)} = \begin{cases} S_{ue}^{(l)}, & e \in \text{Top}_k(S_u^{(l)}) \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

where $W_* \in \mathbb{R}^{d \times N}$ projects the input into the expert embedding space, $S^{(l)} \in \mathbb{R}^{T \times N}$ contains per-token logits over experts, and $\tilde{S}^{(l)} \in \mathbb{R}^{T \times N}$ is a sparse routing matrix in which only the top- k entries per token (per row) are retained and the remaining entries are set to zero. This sparse routing mechanism ensures that, for each token, only a small number of experts are activated according to the current traffic semantics. The output of the sparse branch is the weighted sum of the activated experts:

$$O_{\text{special}}^{(l)} = \sum_{e=1}^N \tilde{S}_{:,e}^{(l)} \odot E_e \left(Z^{(l)} \right), \quad (6)$$

where $\tilde{S}_{:,e}^{(l)}$ denotes routing weights for expert E_e across all tokens and multiplication $\odot$ is broadcast along the feature dimension. By setting $k \ll N$ (e.g., $k = 2$ in this work), *Traffic-MoE* selectively activates parameter subspaces relevant to the specific attack or protocol phase currently being processed.

For the implementation of the experts, both the shared expert and the specialized experts follow a SwiGLU-based gated feed-forward design. Given an input Z , an expert parameterized by $\{W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}\}$ computes:

$$E(Z) = (\text{SiLU}(ZW_{\text{gate}}) \odot (ZW_{\text{up}})) W_{\text{down}}, \quad (7)$$

where $\text{SiLU}(\cdot)$ is the Sigmoid Linear Unit. Note that the shared expert retains a full intermediate dimension $d' \gg d$ for capturing universal protocol patterns (i.e., $W_{\{\text{gate}, \text{up}\}} \in \mathbb{R}^{d \times d'}$ and $W_{\text{down}} \in \mathbb{R}^{d' \times d}$), whereas specialized experts use a reduced intermediate dimension (d'/k) to balance the number of experts and computation (i.e., $W_{\{\text{gate}, \text{up}\}} \in \mathbb{R}^{d \times \frac{d'}{k}}$ and

$\mathbf{W}_{\text{down}} \in \mathbb{R}^{\frac{d'}{k} \times d}$). The weights of different specialized experts are mutually independent.

The output of the MoE layer integrates protocol-invariant features from the shared expert and instance-specific features from the specialized experts, added to the residual stream:

$$\mathbf{H}^{(l)} = \widetilde{\mathbf{H}}^{(l)} + \left(\mathbf{O}_{\text{shared}}^{(l)} + \mathbf{O}_{\text{special}}^{(l)} \right). \quad (8)$$

To prevent routing collapse, where the routing network converges to utilizing only a few experts, leaving others under-trained, we incorporate an auxiliary **load-balancing loss**, $\mathcal{L}_{\text{aux}}$. For a given layer l , we compute the loss based on the batch-wise statistics of expert activation. Let $\text{Load}_e^{(l)}$ be the fraction of tokens in the batch assigned to expert e , and $\text{Prob}_e^{(l)}$ be the average routing probability for expert e across the batch. The load-balancing loss for layer l is:

$$\mathcal{L}_{\text{aux}}^{(l)} = N \sum_{e=1}^N \text{Load}_e^{(l)} \cdot \text{Prob}_e^{(l)}, \quad \mathcal{L}_{\text{aux}} = \frac{1}{L} \sum_{l=1}^L \mathcal{L}_{\text{aux}}^{(l)}. \quad (9)$$

The total auxiliary loss $\mathcal{L}_{\text{aux}}$ is the average of these penalties across all L layers. This constraint ensures a uniform batch-level expert activation distribution, maximizing model's capacity utilization while maintaining sparse inference efficiency.

C. Pre-training and Fine-tuning

To endow *Traffic-MoE* with both broad traffic understanding and task-specific discrimination, we adopt a two-stage training framework consisting of large-scale self-supervised pre-training followed by lightweight few-shot fine-tuning. The pre-training stage exposes the backbone to massive unlabeled traffic data, enabling it to learn protocol-aware and temporally structured representations through autoregressive Next Token Prediction (NTP) and balanced sparse-expert activation. Fine-tuning then adapts these general representations to downstream traffic security applications by employing targeted data augmentation, flow-level semantic pooling, and hierarchical optimization. This two-stage framework enables *Traffic-MoE* to scale in capacity while remaining efficient and robust across diverse traffic security scenarios.

1) Autoregressive Pre-training: Next Token Prediction:

Real-world networks generate massive unlabeled packet flows but only limited annotated security data, making pre-training essential for learning generalizable traffic representations. We adopt a self-supervised autoregressive NTP objective to exploit this abundant unlabeled traffic. As network communication is sequential and governed by protocol-driven state transitions, predicting the subsequent token from historical context enables the model to learn temporal dependencies (e.g., TCP handshake timing) and structural regularities (e.g., TLS record header formats) even in encrypted flows. This process equips *Traffic-MoE* with a unified understanding of network behaviors that can be effectively adapted during downstream fine-tuning. Figure 2 illustrates the pre-training pipeline, highlighting interaction among Traffic2Token representation, causal attention, and sparse experts to form a scalable traffic foundation model.

Given a flow token sequence $X = \{x_1, x_2, \dots, x_T\}$, *Traffic-MoE* learns to maximize the likelihood of the next

token x_t conditioned on its historical context $\mathbf{X}_{<t} = \{x_1, x_2, \dots, x_{t-1}\}$. The prefix $\mathbf{X}_{<t}$ is encoded through L -layer *Traffic-MoE* backbone to produce hidden states $\mathbf{H}_{<t}^{(L)} = \{\mathbf{h}_1^{(L)}, \dots, \mathbf{h}_{t-1}^{(L)}\}$. The final state $\mathbf{h}_{t-1}^{(L)} \in \mathbb{R}^d$ summarizes all contextual information up to position $t-1$ and is projected into the vocabulary space to obtain the probability distribution of next-token prediction:

$$p_\theta(x_t | \mathbf{X}_{<t}) = \text{Softmax} \left(\mathbf{h}_{t-1}^{(L)} \cdot \mathbf{W}_{\text{vocab}} \right), \quad (10)$$

where $\mathbf{W}_{\text{vocab}} \in \mathbb{R}^{d \times |\mathcal{V}|}$ maps hidden states to vocabulary logits. To optimize the model parameters θ , we employ the negative log-likelihood loss function $\mathcal{L}_{\text{NP}}$. For a mini-batch of B flows, this is calculated as the average loss over all valid tokens in the batch via a binary mask $m_t^{(b)} \in \{0, 1\}$:

$$\mathcal{L}_{\text{NP}} = - \frac{1}{\sum_{b=1}^B \sum_{t=2}^T m_t^{(b)}} \sum_{b=1}^B \sum_{t=2}^T m_t^{(b)} \log p_\theta \left(x_t^{(b)} | \mathbf{X}_{<t}^{(b)} \right). \quad (11)$$

Finally, as MoE activates limited experts per token, balanced expert utilization is essential in pre-training. We incorporate load-balancing term $\mathcal{L}_{\text{aux}}$ to prevent routing collapse and promote expert specialization across heterogeneous traffic behaviors. The overall pre-training objective is:

$$\mathcal{L}_{\text{pretrain}} = \mathcal{L}_{\text{NP}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}, \quad (12)$$

where λ_{aux} controls the strength of load-balancing regularization. Minimizing this composite loss enables *Traffic-MoE* to acquire rich, protocol-aware, and temporally grounded representations of traffic flows, while simultaneously encouraging efficient and diverse expert activation. As a result, the pre-trained backbone captures both universal traffic patterns and domain-specific subtleties, providing a strong foundation for downstream tasks such as intrusion detection, service classification, and VPN/Tor traffic analysis.

2) *Few-shot Fine-tuning for Traffic Analysis*: Downstream traffic analysis tasks such as malware detection, service classification and C2 identification typically face label scarcity and highly imbalanced class distributions. To effectively adapt *Traffic-MoE* to these data-constrained scenarios, we introduce a fine-tuning strategy tailored for few-shot learning. This strategy integrates temporal-aware data augmentation, flow-level representation aggregation, and hierarchical optimization to achieve robust performance under limited supervision.

Network sessions vary widely in duration. DNS queries may finish within milliseconds, whereas video streaming or VPN tunnels can persist for minutes or even hours. This discrepancy leads to a long-tailed flow length distribution and extremely limited samples for certain classes. To mitigate this imbalance while precluding data leakage, we apply Temporal Traffic Slicing independently within each partition after a rigorous session-level train/validation/test split, dividing long flows into fixed-duration sub-flows that inherit the original label. The resulting augmented dataset increases sample diversity and enables the model to capture fine-grained temporal dynamics such as periodic beacons, idle intervals, and bursty transmissions that are often diluted in full-session representations. This

augmentation maximizes the utility of sparse session-level examples and is particularly effective in few-shot scenarios.

For supervised adaptation, we aggregate token-level representations from the final backbone layer into a unified flow-level embedding. Instead of relying on a single special token (e.g., [PD] or [END]), which may overlook mid-session structural semantics, we apply mean pooling aggregation over all valid tokens (indicated by the binary mask m_t) and compute class probabilities through a MLP-based prediction head:

$$p_\theta(y | X) = \text{Softmax} \left(\text{MLP} \left(\frac{1}{\sum_{t=1}^T m_t} \sum_{t=1}^T m_t \mathbf{h}_t^{(L)} \right) \right). \quad (13)$$

For a mini-batch of B flow token sequences $\{X^{(b)}\}_{b=1}^B$ with labels $\{y_b\}_{b=1}^B$, the supervised objective is defined as the batch-averaged cross-entropy loss:

$$\mathcal{L}_{\text{TC}} = -\frac{1}{B} \sum_{b=1}^B \log p_\theta(y_b | X^{(b)}). \quad (14)$$

To maintain stable expert routing and prevent expert collapse during adaptation, we retain the MoE load-balancing term $\mathcal{L}_{\text{aux}}$, yielding the overall fine-tuning objective:

$$\mathcal{L}_{\text{finetune}} = \mathcal{L}_{\text{TC}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}. \quad (15)$$

By minimizing this composite loss, *Traffic-MoE* can precisely adapt the general traffic knowledge learned during pre-training to downstream classification tasks while efficiently leveraging its powerful MoE architecture, thereby continuously improving its classification performance in specific traffic security applications.

Finally, to mitigate catastrophic forgetting of pre-trained knowledge, we employ a Layer-wise Learning Rate Decay (LLRD) schedule tailored to the hierarchical structure of *Traffic-MoE*. Specifically, shallower layers encoding generic packet syntax are updated conservatively with lower learning rates. In contrast, deeper layers and the classification head have higher learning rates, facilitating quick adaptation to task-specific semantics. For a backbone with L layers and base learning rate η_0 , the learning rate for layer l is:

$$\eta_l = \xi^{L-l} \cdot \eta_0, \quad (16)$$

where $\xi < 1$ is the decay factor. This strategy preserves the universal traffic knowledge encoded during pre-training while enabling the deeper layers to adjust to the semantic and distributional characteristics of the downstream task.

By combining temporal slicing, flow-level semantic pooling, sparse-expert regularization, and hierarchical optimization, *Traffic-MoE* achieves high sample efficiency and robust generalization in data-constrained traffic security scenarios. Even with limited labeled examples, *Traffic-MoE* adapts effectively to new domains and evolving threats while retaining the broad traffic knowledge learned during pre-training.

V. EXPERIMENT EVALUATION

A. Datasets and Downstream Tasks

1) *Pre-training Corpus*: To facilitate the learning of universal and robust traffic representations across heterogeneous

network environments, we construct a comprehensive pre-training corpus aggregating over **2 million unlabeled flows** from five authoritative public datasets (*USTC-TFC2016* [39], *UNSW-NB15* [40], *CICIoT2023* [41], *CICIoMT2024* [42], *ISCXVPN2016(NonVPN)* [43]). These datasets encompass a wide spectrum of encryption complexities, protocol diversities, and behavioral signatures. Unlike prior works restricted to domain-specific subsets, our corpus incorporates extensive *distributional diversity* by intertwining standard IT traffic (e.g., HTTP/DNS) with specialized OT protocols (e.g., MQTT/Zigbee) and heavily obfuscated tunnels. This cross-domain dataset synthesis forces the model to learn universal and intrinsic traffic semantics rather than dataset-specific artifacts.

2) *Fine-tuning Datasets and Downstream Tasks*: During fine-tuning phase, we select five challenging public datasets to design four downstream tasks, systematically evaluating model’s transfer learning capability from attack detection to service classification: **① For Tor/NonTor Service Classification**, we leverage *ISCXTor2016* [44] to classify 8 application services within separated “Tor” and “NonTor” traffic subsets, challenging the model to capture subtle flow-level fingerprints in extreme anonymization scenarios; **② For NonVPN/Mixed Service Classification**, we employ *ISCXVPN2016* to construct two sub-tasks, “NonVPN” and “Mixed” service classification, assessing the restoration of service semantics under tunnel encapsulation; **③ For IoT/IoMT Attack Detection**, we utilize *CICIoT2023* to identify 33 distinct attack behaviors, and *CICIoMT2024* to detect 18 targeted attacks, testing performance in sensitive healthcare environments involving protocols like MQTT. **④ For Modern Encrypted Service Classification**, we employ *CipherSpectrum* [45] and group domains into 10 typical service categories, validating the model’s ability to distinguish fine-grained services under pure TLS 1.3 encryption. Detailed descriptions are provided in Table I and Appendix B.

B. Comparison Methods and Parameter Setup

1) *Comparison Methods*: To precisely benchmark *Traffic-MoE*, we evaluate it against eight SOTA baselines categorized into two paradigms. As core baselines representing the “pre-training and fine-tuning” paradigm, we select *ET-BERT* [11], *NetGPT* [12], *TrafficFormer* [13] *YaTC* [25], and *NetMamba* [26] for comparison. Additionally, to verify the effectiveness of the pre-training phase, we include two ML methods (*AppScanner* [19], *FlowPrint* [20]) and one DL method (*FS-Net* [10]) for comparison. To ensure fairness, all comparison models are reproduced based on their official open-source code. Specifically, the pre-training methods utilize the exact same pre-training and fine-tuning datasets constructed in this work, adopting identical early stopping strategies during fine-tuning to ensure optimal convergence, while the ML/DL methods are trained directly on the constructed fine-tuning datasets. More details of comparison methods are presented in Appendix C.

2) *Implementation Details and Parameter Configuration*: All experiments are implemented in PyTorch 2.2 and executed on a high-performance cluster equipped with Intel Xeon Gold 5218R CPUs and NVIDIA A100 GPUs (40GB VRAM). The environment runs on Ubuntu 22.04 LTS with CUDA 11.7.

TABLE I: Overview of datasets used in pre-training and fine-tuning phases

Dataset	Domain	Original	Original Scale [*]	Key Protocols	Pre-training		Fine-tuning			
		Classes	(Record)		Included	Size (Sessions)	Included	Downstream Task	Size (Sessions)	Classes
USTC-TFC2016	Malware Traffic	20	0.7M rows	FTP, SMB, HTTP, DNS, MySQL, ...	✓	203.2k	✗	-	-	-
UNSW-NB15	Network Intrusion	10	2.5M rows	HTTP, ICMP, FTP, SSH, DNS, ...	✓	828.3k	✗	-	-	-
CICIoT2023	IoT Security	34	46.7M rows	MQTT, HTTP, DNS, SSH, ARP, ...	✓	892.5k	✓	IoT Attack Detection	255.2k	34
CICIoMT2024	IoMT Security	19	8.8M rows	Wi-Fi, MQTT, HTTP, DNS, SSH, ...	✓	439.4k	✓	IoMT Attack Detection	107.3k	19
ISCXVPN2016(NonVPN)	Unencrypted Traffic	14	298.9k sessions	OpenVPN, BitTorrent, SFTP, FTPS, HTTPS, SMTPS, POP3S, ...	✓	160.2k	✓	NonVPN Traffic Detection	23.9k	6
ISCXVPN2016(Mixed)	VPN Encrypted Traffic						✗	-	✓	Mixed Traffic Detection
ISCTXor2016(NonTor)	Unencrypted Traffic	8	53.7k sessions	Tor, BitTorrent, HTTPS, SMTP, POP3, SFTP, FTPS, ...	✗	-	✓	NonTor Traffic Detection	51.5k	8
ISCTXor2016(Tor)	Tor Encrypted Traffic						✗	-	✓	Tor Traffic Detection
CipherSpectrum	TLS Encrypted Traffic	40	120.0k sessions	TLS-AES-128-GCM, TLS-AES-256-GCM, TLS-CHACHA20-POLY1305	✗	-	✓	TLS Traffic Detection	41.2k	10

* Note: The unit “rows” refers to pre-processed data records (e.g., packet windows or flow images) as defined in the original datasets, whereas “sessions” refers to complete bidirectional flows reconstructed from raw PCAP traces. For datasets appearing in both pre-training and fine-tuning, the sessions allocated to each phase are strictly non-overlapping.

Data Preprocessing and Serialization: PCAP traces are first segmented into independent sessions via SplitCap. For datasets utilized in both pre-training and fine-tuning phases, these sessions are rigorously partitioned into two mutually exclusive subsets to ensure the complete exclusion of fine-tuning data from the pre-training corpus, prior to independent serialization via *Traffic2Token*. Specifically, we extract 11 bytes of header features and the first $J = 40$ bytes of payload per packet, truncating each session to the first $K = 10$ packets to balance computational efficiency and early-stage detection.

Training Configuration: We utilize AdamW optimizer with a batch size of $B = 32$. **For the pre-training phase**, the model is trained for 8 epochs with a learning rate of 3.0×10^{-4} . The loss function combines the NTP loss and the auxiliary load-balancing loss, with the regularization coefficient set to $\lambda_{\text{aux}} = 0.02$. **For the fine-tuning phase, we utilize session samples that are strictly disjoint from the pre-training corpus to prevent data leakage.** These data are split into training, validation, and testing sets with an 8:1:1 ratio. We employ a LLRD schedule to preserve pre-trained knowledge, setting the decay factor $\xi = 0.9$ and the base learning rate $\eta_0 = 5.0 \times 10^{-5}$. The maximum training duration is set to 40 epochs, safeguarded via early stopping with a patience of 5 epochs monitoring the validation Macro-F1 score.

3) *Evaluation Metrics:* Given the inherent long-tail distribution and class imbalance in network traffic, standard accuracy is insufficient for objective evaluation. We therefore adopt a multi-dimensional evaluation protocol. For evaluating general performance, we select Macro-Precision (M-PR), Macro-Recall (M-RC), and Macro-F1 Score (M-F1) as core metrics. By calculating performance independently for each class before averaging, these macro-metrics treat minority classes (e.g., emerging threats) and majority classes equally, preventing results from being biased by high-volume benign traffic. For evaluating security effectiveness, we specifically report the False Negative Rate (FNR) and False Positive Rate (FPR). These metrics are critical for assessing the operational risks of the model, quantifying the trade-off between missed threats and false alarms in real-world deployments.

C. Comparison with State-of-the-Art Methods

Table II reports the overall performance of all methods across diverse traffic analysis scenarios, where *Traffic-MoE* consistently demonstrates highly competitive performance

across all evaluation metrics. A cross-paradigm comparison reveals a substantial performance gap: statistical ML methods and DL models trained from scratch lag considerably behind pre-trained methods on most tasks. This disparity highlights a fundamental challenge in modeling heterogeneous network traffic. Traditional methods relying on shallow statistical features struggle to capture complex protocol behaviors obscured by encryption. Similarly, training deep networks from scratch fails to infer intricate protocol state transitions and temporal burst patterns given the scarcity of labeled attack samples. In contrast, by leveraging self-supervised pre-training on a massive unlabeled traffic corpus, *Traffic-MoE* internalizes the intrinsic “grammar” of network protocols (e.g., handshake sequences and payload structures), providing a semantic-rich initialization that significantly reduces dependency on task-specific labeled traffic data.

In obfuscated Tor encrypted scenarios, *Traffic-MoE* demonstrates exceptional robustness. As shown in the *ISCTXTor2016* results, *Traffic-MoE* achieves Macro-F1 scores of 0.8900 (NonTor) and 0.8072 (Tor), outperforming the strongest baseline by 4.68% and 11.28%, respectively. Notably, existing strong baselines such as ET-BERT and NetMamba suffer from noticeable performance degradation in Tor traffic. This decline stems from Tor protocol’s multi-layer encryption mechanisms, which introduce significant structural noise, thereby confusing dense architectures that process all tokens uniformly. *Traffic-MoE* overcomes this challenge through its sparse MoE architecture. The dynamic routing mechanism allows specific experts to specialize in decoupling valid behavioral fingerprints from padding noise, thereby maintaining high discriminative power even under extreme anonymity constraints.

Similarly, in VPN-encapsulated environments, *Traffic-MoE* effectively penetrates protocol obfuscation to identify underlying services. On the *ISCXVPN2016* dataset, particularly in the “Mixed Traffic” detection task, *Traffic-MoE* outperforms the generative pre-trained model NetGPT by 10.12% in accuracy and 3.71% in Macro-F1. While traditional methods like FlowPrint struggle due to the homogenization of observable statistical features by VPN tunneling, *Traffic-MoE* succeeds by modeling long-range contextual dependencies. Through the self-attention mechanism, it captures intrinsic flow evolution patterns that remain invariant across tunnel encapsulation, thus resisting the interference of outer protocol headers.

Regarding attack detection, *Traffic-MoE* exhibits critical

TABLE II: Comparison with state-of-the-art methods on different traffic detection tasks

Method	Task 1: ISCXTor2016(NonTor)				Task 1: ISCXTor2016(Tor)				Task 2: ISCXVPN2016(NonVPN)				Task 2: ISCXVPN2016(Mixed)			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
AppScanner	0.9468	0.8587	0.6934	0.7436	<u>0.8690</u>	0.7921	0.7054	<u>0.7254</u>	0.5794	0.6756	0.6782	0.6588	0.6249	0.7414	0.7355	0.7248
FlowPrint	0.9213	0.7408	0.7245	0.7138	0.4127	0.1907	0.2595	0.1738	0.5459	0.6283	0.5620	0.5617	0.5659	0.6033	0.6431	0.5952
FS-Net	0.9406	0.7841	0.6518	0.6972	0.6679	0.2008	0.2527	0.2161	0.5093	0.4952	0.5021	0.4971	0.5357	0.4617	0.4681	0.4631
ET-BERT	0.9653	0.8322	0.7706	0.7960	0.7932	0.4093	0.4677	0.4303	0.5502	0.6670	0.6852	0.6755	0.6106	0.7503	0.7549	0.7521
NetGPT	0.9655	0.8479	0.7911	0.8134	0.8506	0.7342	0.6805	0.6583	0.6618	0.7405	0.7511	0.7457	0.6973	0.8003	<u>0.8072</u>	<u>0.8034</u>
TrafficFormer	0.9554	0.7663	0.7596	0.7418	0.7975	0.6083	0.5984	0.5428	0.6045	0.6831	0.6970	0.6842	0.6373	0.7689	<u>0.7285</u>	<u>0.7422</u>
YaTC	<u>0.9736</u>	0.9001	0.8203	<u>0.8502</u>	0.8621	<u>0.8028</u>	<u>0.7565</u>	0.7106	<u>0.6860</u>	<u>0.7550</u>	0.7494	0.7207	<u>0.7022</u>	<u>0.8036</u>	0.7814	0.7849
NetMamba	0.9715	0.8554	<u>0.8287</u>	0.8414	0.7488	0.5550	0.4856	0.4892	0.6417	0.6904	0.6543	0.6622	0.6605	0.7117	0.6760	0.6756
Traffic-MoE	0.9827	0.9222	0.8707	0.8900	0.9089	0.8942	0.7879	0.8072	0.7613	0.7866	0.8158	0.8005	0.7679	0.8306	0.8377	0.8332

Method	Task 3: CICIoMT2024				Task 3: CICIoT2023				Task 4: CipherSpectrum			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
AppScanner	0.6929	0.7473	0.6783	0.6397	0.5266	0.5535	0.4471	0.4619	<u>0.9276</u>	<u>0.9248</u>	0.9199	<u>0.9270</u>
FlowPrint	0.0132	0.0373	0.1364	0.0501	0.4578	0.3166	0.2048	0.1989	0.2900	0.2580	0.2702	0.2602
FS-Net	0.5788	0.5319	0.5050	0.4533	0.4903	0.5265	0.4046	0.4094	0.8714	0.8447	0.8359	0.8388
ET-BERT	0.9769	0.5620	0.5271	0.5255	0.7455	0.5416	0.5172	0.5217	0.4630	0.4267	0.3618	0.3474
NetGPT	0.8958	0.8326	0.8335	0.8300	0.7684	0.7756	0.6709	0.6962	0.4503	0.4362	0.3860	0.3813
TrafficFormer	0.8912	0.7572	0.7835	0.7530	0.7636	0.7172	0.6411	0.6594	0.4564	0.4296	0.3539	0.3568
YaTC	0.9635	0.8578	0.8683	0.8503	0.8747	0.8084	0.7661	0.7745	0.8994	0.9241	0.8993	0.9143
NetMamba	0.9626	0.8756	0.8531	<u>0.8638</u>	0.8520	0.8122	<u>0.7675</u>	0.7801	0.6725	0.8886	0.6155	0.7018
Traffic-MoE	0.9769	0.8849	0.8860	0.8839	<u>0.8588</u>	0.8007	0.7701	0.7824	0.9305	0.9321	0.9323	0.9307

advantages in identifying malicious activities within massive background traffic. On the *CICIoMT2024* dataset, *Traffic-MoE* achieves an accuracy of 0.9769 and outperforms the second-best model by 2.33% in Macro-F1. On the more complex *CICIoT2023* dataset, which comprises 34 categories, *Traffic-MoE* establishes leading Macro-F1 (0.7824) and Macro-Recall (0.7701). Although Accuracy and Macro-Precision marginally trail YaTC and NetMamba, *Traffic-MoE* strikes a superior precision-recall trade-off. This indicates that our MoE-driven design effectively mitigates the bias toward majority classes, enabling specialized experts to capture fine-grained nuances of minority attack variants in the long-tail distribution. Consequently, *Traffic-MoE* yields a more comprehensive detection capability across diverse threat categories. Furthermore, we observe that traditional methods fail to maintain performance in this scenario due to the severe class imbalance and the presence of diverse attack variants. Pre-trained models address this by internalizing anomalous patterns during the pre-training phase, enabling them to generalize from common attacks to unseen variants and effectively mitigate the long-tail distribution issue inherent in real-world threat detection.

Notably, *Traffic-MoE* demonstrates superior resilience on *CipherSpectrum*, a challenging pure TLS 1.3 encrypted traffic dataset. While existing pre-trained models, such as ET-BERT, NetGPT, and TrafficFormer, experience severe feature collapse on this dataset (with Macro-F1 plummeting below 0.39), *Traffic-MoE* establishes a new SOTA, achieving an accuracy of 0.9305 and a Macro-F1 of 0.9307. This performance surpasses both generic pre-trained baselines and statistical fingerprinting methods, validating *Traffic-MoE*'s capacity to effectively overcome representation bottlenecks introduced by modern, highly encrypted traffic environments.

D. Comparison in Few-Shot Settings

In real-world cybersecurity operations, acquiring large-scale, high-quality labeled traffic is often cost-prohibitive due

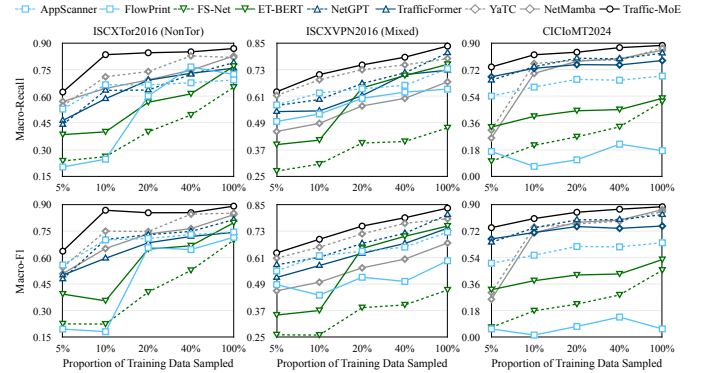


Fig. 5: Performance evaluation in few-shot supervision.

to the rapid evolution of attack variants. To evaluate robustness under *Label Scarcity*, we conduct a stratified few-shot evaluation on three representative datasets (*ISCTXTor2016-NonTor*, *ISCXVPN2016-Mixed*, *CICIoMT2024*), scaling training data from 5% to 100%. The performance trajectories in Figure 5 reveal three distinct evolution paradigms. Scratch-trained DL methods (e.g., FS-Net) exhibit data-hungry collapse, where the lack of prior knowledge leads to parameter non-convergence and precipitous performance drops (over 85% decline) in extreme few-shot settings. In contrast, statistical ML methods (e.g., AppScanner) demonstrate stability but suffer from a heuristic representation ceiling, failing to scale performance with increased data due to the limited expressiveness of hand-crafted features. Most notably, *Traffic-MoE* achieves **sample-efficient transfer**, significantly outperforming baselines at the 5% data extreme and often matching the full-data performance of competitors with only 10%-20% supervision, validating that the pre-trained MoE architecture successfully internalizes transfer-ready universal traffic semantics.

A deeper inspection of critical crossovers in Figure 5 highlights the architectural advantage of sparse experts over dense transformers. In complex encrypted scenarios (*IS-*

TABLE III: Ablation study results under different traffic detection tasks

Ablation Variant	Task 1: ISCXTor2016(NonTor)				Task 2: ISCXVPN2016(Mixed)				Task 3: CICIoMT2024				Task 4: CipherSpectrum			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
Traffic-MoE(Ours)	0.9827	0.9222	0.8707	0.8900	0.7679	0.8306	0.8377	0.8332	0.9769	0.8849	0.8860	0.8839	0.9305	0.9321	0.9323	0.9307
MoE→Dense(Total)	0.9831	0.9229	0.8655	0.8904	0.7600	0.8101	0.8302	0.8193	0.9790	0.8957	0.8947	0.8946	0.9300	0.9346	0.9233	0.9280
MoE→Dense(Active)	0.9806	0.9069	0.8540	0.8758	0.7372	0.7959	0.8011	0.7976	0.9760	0.8783	0.8787	0.8774	0.9249	0.9284	0.9168	0.9222
w/o Shared Expert	0.9725	0.8814	0.7854	0.8182	0.6636	0.7088	0.7220	0.7143	0.9110	0.8332	0.8344	0.8221	0.7149	0.7438	0.6984	0.7116
Dense(Total) w/o PT	0.9674	0.8064	0.8054	0.7938	0.6111	0.6958	0.6680	0.6770	0.9728	0.8350	0.8346	0.8294	0.8577	0.8694	0.8499	0.8584
w/o PT	0.9643	0.7753	0.7271	0.7481	0.6255	0.6883	0.6865	0.6854	0.9685	0.8110	0.8236	0.8154	0.6833	0.6842	0.6601	0.6635
w/o $\mathcal{L}_{aux}$	0.9800	0.8786	0.8658	0.8718	0.7353	0.8064	0.8205	0.8107	0.9767	0.8674	0.8725	0.8692	0.9240	0.9222	0.9162	0.9185
w/o Traffic2Token	0.9716	0.9009	0.8403	0.8719	0.6715	0.7663	0.7844	0.7732	0.9722	0.8249	0.8216	0.8109	0.9126	0.9088	0.9074	0.9079
Traffic-MoE(Ours)	0.9827	0.9222	0.8707	0.8900	0.7679	0.8306	0.8377	0.8332	0.9769	0.8849	0.8860	0.8839	0.9305	0.9321	0.9323	0.9307
w/ Header Only	0.9750	0.8776	0.7983	0.8290	0.7425	0.8050	0.8150	0.8089	0.9263	0.8444	0.8539	0.8481	0.9271	0.9224	0.9253	0.9226
w/ Payload Only	0.8317	0.7535	0.7209	0.7455	0.5199	0.7147	0.5382	0.5648	0.4879	0.4603	0.4459	0.4210	0.5390	0.5175	0.4991	0.4897
w/o Payload-Data	0.9825	0.9097	0.8684	0.8884	0.7653	0.8345	0.8336	0.8401	0.9813	0.8831	0.8912	0.8797	0.9296	0.9323	0.9343	0.9301
w/o Payload-Negotiation	0.9096	0.8077	0.7778	0.8064	0.6822	0.7437	0.6709	0.6916	0.8273	0.7536	0.7471	0.7483	0.5893	0.6964	0.7190	0.6449
w/o Header-Length	0.9544	0.8753	0.7741	0.7866	0.6434	0.7140	0.6568	0.6727	0.8914	0.6968	0.7247	0.6794	0.5382	0.5724	0.5027	0.4991
w/o Header-Direction	0.9750	0.8679	0.8290	0.8412	0.7387	0.8115	0.7862	0.7949	0.9620	0.7498	0.7378	0.7434	0.9225	0.9278	0.9172	0.9208
w/o Header-Flags	0.9690	0.8445	0.8559	0.8450	0.7227	0.7919	0.7472	0.7668	0.7848	0.7119	0.6579	0.6443	0.9106	0.9205	0.9019	0.9047
w/o Header-Proto	0.9808	0.8966	0.8580	0.8741	0.7638	0.8305	0.8297	0.8309	0.9701	0.7855	0.7702	0.7773	0.9245	0.9308	0.9284	0.9268
w/o Header-Time	0.9366	0.7722	0.7892	0.7658	0.5632	0.7754	0.6859	0.6599	0.7426	0.5617	0.5500	0.5264	0.5676	0.8503	0.5681	0.5825

CSXVPN2016), we observe a watershed at the 20% data ratio: below this threshold, dense pre-trained baselines (e.g., ET-BERT) suffer significant degradation, dropping below even traditional ML methods. This indicates that dense architectures are prone to *overfitting and representation collapse* when fine-tuning data is insufficient to constrain their massive parameter space. Conversely, *Traffic-MoE* maintains highly competitive performance, suggesting that *sparse expert activation mechanism effectively acts as a regularizer*. By activating only a subset of parameters ($k = 2$) per token, it reduces effective model complexity for specific tasks to prevent overfitting. Furthermore, in intrusion detection tasks (CICIoMT2024), *Traffic-MoE* maintains robust performance even when traditional methods collapse, demonstrating significant **generalization** potential for identifying emerging threats by effectively transferring learned protocol semantics to attack signatures.

E. Ablation Analysis

To quantify the contributions of *Traffic-MoE*’s key architectural designs and input flow sequence features, we conduct a multi-dimensional ablation study (results for four representative tasks are summarized in Table III, with more dataset evaluations detailed in Appendix F).

1) *Effectiveness of Key Model Components*: We construct model variants by replacing or removing critical modules to validate the architectural rationality of *Traffic-MoE*. **① Sparse Routing and Expert Specialization** Compared to parameter-matched dense baselines, *Traffic-MoE* achieves superior performance while maintaining extremely low inference overhead. Specifically, compared to the active-parameter-matched variant “MoE→Dense(Active)”, it yields an average Macro-F1 improvement of 1.94%. In the complex ISCXVPN2016(Mixed) scenario, it even outperforms the total-parameter-matched variant “MoE→Dense(Total)” by a margin of 1.70%. This substantiates that dynamic routing effectively decouples heterogeneous traffic patterns, successfully breaking through the representation bottlenecks inherent in dense architectures. Furthermore, eliminating the persistently activated shared expert (w/o Shared Expert) induces a consistent performance degradation, particularly in encrypted traffic tasks. This confirms its critical role as an anchor for universal protocol semantics, preventing

the representational capacity of specialized experts from being diluted. **② Necessity of the Pre-training Paradigm**: Removing the pre-training phase (w/o PT) causes an average Macro-F1 plummet of 17.54% across all tasks. Notably, without pre-training, the performance degradation of the MoE architecture far exceeds that of dense models (e.g., dropping by 28.71% vs. 7.50% in TLS encrypted scenarios). This stark contrast reveals a critical mechanism: the routing module heavily relies on a robust, pre-established semantic space. Only upon this foundation can the router avoid blind allocation and converge to an optimal state of expert specialization. **③ Load Balancing and Data Processing Pipeline**: Removing the load-balancing loss $\mathcal{L}_{aux}$ results in a general Macro-F1 decline of 1.31% to 2.70%, indicating that $\mathcal{L}_{aux}$ is essential for ensuring balanced expert activation and maximizing the utilization of overall model capacity. Furthermore, compared to directly feeding raw traffic bytes, employing the Traffic2Token processing pipeline yields an average Macro-F1 improvement of 5.34%, proving that this design effectively mitigates the learning difficulty associated with long sequence features.

2) *Effectiveness of Traffic Sequence Components*: To explore the contributions of multimodal features, we perform fine-grained component masking and disentanglement on the input flow sequences: **① Modality Complementarity and Phase Awareness**: Macroscopically, header metadata dominates in most tasks (i.e., w/ Header Only significantly outperforms w/ Payload Only), while *Traffic-MoE* successfully compensates for single-modality deficiencies through multi-view fusion. Deeper “payload phase awareness” experiments reveal that masking the *data transfer phase* of the payload (w/o Payload-Data) has a negligible impact on performance; however, masking the *handshake negotiation phase* (w/o Payload-Negotiation) triggers a sharp Macro-F1 drop of 9.39% to 30.71%. This provides conclusive evidence: the performance gain derived from payload features essentially originates from the plaintext interaction structures during handshakes, rather than the semantically opaque encrypted application data. **② Metadata Feature Importance Quantification**: Quantitative results from masking individual header metadata attributes (using [PAD]) demonstrate that the packet inter-arrival time contributes the most (an average Macro-F1 drop of 28.15%

when removed), whereas the protocol type (Proto) has the minimal impact. Crucially, in highly obfuscated modern encryption scenarios, the discriminative power of length features is significantly amplified; removing these features leads to a catastrophic Macro-F1 degradation of 46.37%.

F. Inference Efficiency Analysis

To rigorously assess efficiency, we conduct a comprehensive inference efficiency analysis of *Traffic-MoE* against the total-parameter-matched dense variant (MoE→Dense) and other competitors on *CICIoMT2024* across varying concurrency loads (batch size $B \in \{8, 16, 32, 64\}$). As illustrated in Figure 6, *Traffic-MoE* exhibits a pronounced efficiency advantage that scales non-linearly with traffic load. Specifically, under high-load conditions ($B = 64$), *Traffic-MoE* achieves a 41.57% improvement in throughput and a 29.36% reduction in latency compared to the dense variant. This gain stems from decoupling model capacity from inference FLOPs. By dynamically routing tokens to only the top-2 experts, *Traffic-MoE* reduces computational complexity from $\mathcal{O}(N_{\text{total}})$ to $\mathcal{O}(N_{\text{active}})$, effectively bypassing approximately 40% of redundant computation. Furthermore, while the runner-up NetMamba inherently exhibits lower peak memory consumption due to underlying operator optimizations, *Traffic-MoE* leverages *activation sparsity* to achieve a vastly superior latency-throughput trade-off, delivering a 70.42% increase in throughput and a 41.39% decrease in average latency. This demonstrates that our MoE-driven design effectively circumvents the traditional computational bottlenecks of dense architectures, making it exceptionally well-suited for deployment on high-speed, throughput-sensitive network security gateways.

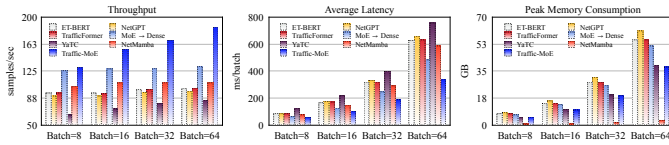


Fig. 6: Inference efficiency comparison on *CICIoMT2024*.

G. Visual Analysis of Expert Activation Preferences

To qualitatively reveal the domain specialization of individual experts for specific traffic patterns within the MoE architecture, we perform a visual analysis of the traffic category-to-expert mapping in the deeper layers of the model (as illustrated in Figure 7). Since shallow layers tend to extract generic protocol grammars, our analysis focuses on the deep layers where task-specific specialization occurs.

For quantitative assessment, instead of using raw Softmax probabilities, we compute the empirical activation frequency of each expert across the dataset. To eliminate distribution bias from the global routing baseline, we further calculate the Relative Preference Rate, defined as the relative deviation of category-specific activation frequency from the global average. As indicated by the color scale, warm tones and cool tones quantify an expert’s positive preference (over-activation) or active repulsion toward a specific traffic category, respectively.

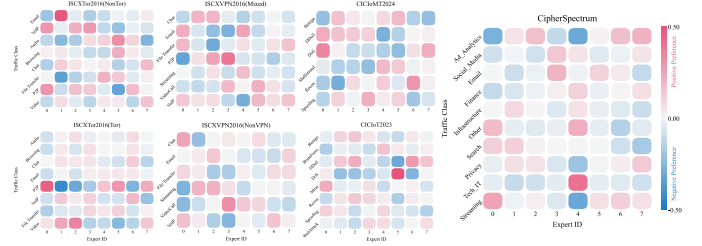


Fig. 7: Expert activation preference across detection tasks.

As shown in Figure 7, the fine-tuned model exhibits highly differentiated routing patterns across four distinct traffic scenarios. The unique combinations of preferred and repelled experts for different categories substantiate that the routing network has learned to dynamically and precisely invoke specific expert subsets based on the intrinsic semantics of the input traffic. Furthermore, while minor expert overlap exists between categories within homogeneous datasets, this phenomenon reflects the model’s strategic parameter reuse and collaborative feature extraction within a constrained parameter space. Further analysis reveals that a single traffic category typically triggers positive preferences across multiple experts. This collaborative mechanism is facilitated by the auxiliary load-balancing loss $\mathcal{L}_{\text{aux}}$, which prevents expert saturation and encourages diverse experts to jointly model complex traffic representations. Consequently, the model effectively avoids “expert collapse” and leverages the sparse parameter space for multi-faceted feature decomposition.

Overall, the visual evidence confirms that experts within MoE layers autonomously evolve into domain experts specialized in downstream task features. By capturing high-level behavioral semantics (e.g., encryption entropy distributions, traffic burst patterns), these experts significantly expand the model’s decision boundary in complex network environments while maintaining robust generalization.

H. Evaluation under Distribution Shifts

Real-world network environments are highly dynamic, characterized by continuous temporal evolution and non-stationary class distributions. To evaluate whether *Traffic-MoE* can adapt to such “open-world” challenges without frequent retraining, we construct three OOD benchmarks: **Time-shift** (simulating concept drift over time), **Proportion-shift** (simulating intra-class distribution shift), and **Compose-shift** (simulating unseen attack sub-variants). Please refer to Appendix I for more details. Figure 8 visualizes the performance comparison across these scenarios. **① Resilience to Concept Drift:** The most significant performance gap appears in the Time-shift scenario. While existing competitors (e.g., FS-Net, ET-BERT) suffer catastrophic degradation due to temporal feature shifts, *Traffic-MoE* maintains a robust performance, outperforming the runner-up by 32.31%–60.18% across datasets. This suggests that the sparse expert architecture effectively disentangles time-invariant protocol semantics from transient statistical noise, preventing the model from overfitting to temporal shortcuts. **② Stability under Structural Shifts:** In

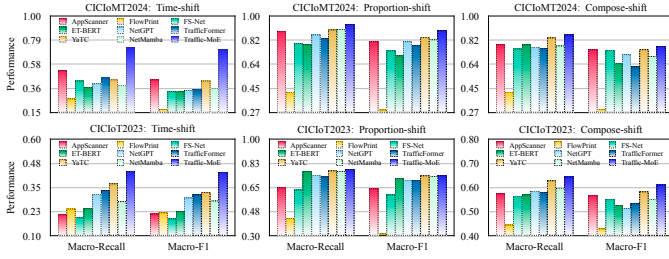


Fig. 8: Performance under different distribution shifts.

scenarios involving structural mutations (Proportion/Compose-shift), *Traffic-MoE* consistently achieves competitive performance. Notably, in the Compose-shift task (where 50% of sub-classes are unseen during training), *Traffic-MoE* surpasses strong pre-trained competitors (e.g., NetGPT, YaTC) by significant margins. High recall rates in these tasks indicate that the model learns generalized behavioral abstractions rather than memorizing specific signatures of seen sub-classes. In summary, *Traffic-MoE* exhibits superior generalization and robustness against distribution shifts, validating its viability for sustained operation in evolving network defense systems.

VI. CONCLUSION

This work bridges the gap between advanced representation learning and practical network protection by introducing *Traffic-MoE*, an efficient sparse foundation model. By synergizing self-supervised pre-training with a dynamic MoE architecture, *Traffic-MoE* effectively decouples model capacity from inference overhead. Extensive evaluations demonstrate that *Traffic-MoE* establishes highly competitive performance across heterogeneous security tasks including intrusion detection and encrypted service classification while significantly reducing inference latency compared to dense competitors. Furthermore, its superior robustness and generalization under few-shot supervision and distribution shifts confirms that sparse foundation models offer a scalable, resilient, and efficient paradigm for next-generation real-time network defense.

REFERENCES

- [1] A. Langley, A. Riddoch, A. Wilk, A. Vicente, C. Krasic, D. Zhang, F. Yang, F. Kouranov, I. Swett, J. Iyengar *et al.*, “The quic transport protocol: Design and internet-scale deployment,” in *Proceedings of the conference of the ACM special interest group on data communication*, 2017, pp. 183–196.
- [2] M. A. Al-Garadi, A. Mohamed, A. K. Al-Ali, X. Du, I. Ali, and M. Guizani, “A survey of machine and deep learning methods for internet of things (iot) security,” *IEEE communications surveys & tutorials*, vol. 22, no. 3, pp. 1646–1685, 2020.
- [3] E. Papadogiannaki and S. Ioannidis, “A survey on encrypted network traffic analysis applications, techniques, and countermeasures,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 6, pp. 1–35, 2021.
- [4] B. Anderson and D. McGrew, “Identifying encrypted malware traffic with contextual flow data,” in *Proceedings of the 2016 ACM workshop on artificial intelligence and security*, 2016, pp. 35–46.
- [5] M. Shen, J. Wu, K. Ye, K. Xu, G. Xiong, and L. Zhu, “Robust detection of malicious encrypted traffic via contrastive learning,” *IEEE Transactions on Information Forensics and Security*, 2025.
- [6] Y. Zeng, H. Gu, W. Wei, and Y. Guo, “deep – full – range: a deep learning based network encrypted traffic classification and intrusion detection framework,” *IEEE Access*, vol. 7, pp. 45 182–45 190, 2019.

- [7] I. Fosić, D. Žagar, K. Grgić, and V. Križanović, “Anomaly detection in netflow network traffic using supervised machine learning algorithms,” *Journal of industrial information integration*, vol. 33, p. 100466, 2023.
- [8] M. Roughan, S. Sen, O. Spatscheck, and N. Duffield, “Class-of-service mapping for qos: a statistical signature-based approach to ip traffic classification,” in *Proceedings of the 4th ACM SIGCOMM conference on Internet measurement*, 2004, pp. 135–148.
- [9] A. Azab, M. Khasawneh, S. Alrabae, K.-K. R. Choo, and M. Sarsour, “Network traffic classification: Techniques, datasets, and challenges,” *Digital Communications and Networks*, vol. 10, no. 3, pp. 676–692, 2024.
- [10] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, “Fs-net: A flow sequence network for encrypted traffic classification,” in *IEEE INFOCOM 2019-IEEE Conference On Computer Communications*. IEEE, 2019, pp. 1171–1179.
- [11] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, “Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification,” in *Proceedings of the ACM Web Conference*, 2022, pp. 633–642.
- [12] X. Meng, C. Lin, Y. Wang, and Y. Zhang, “Netgpt: Generative pretrained transformer for network traffic,” *arXiv preprint arXiv:2304.09513*, 2023.
- [13] G. Zhou, X. Guo, Z. Liu, T. Li, Q. Li, and K. Xu, “Trafficformer: An efficient pre-trained model for traffic data,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1844–1860.
- [14] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations*, 2017, pp. 1–19.
- [15] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [16] G. Wei and Z. Wang, “Adoption and realization of deep learning in network traffic anomaly detection device design,” *Soft Computing-A Fusion of Foundations, Methodologies & Applications*, vol. 25, no. 2, pp. 1147–1158, 2021.
- [17] A. W. Moore and K. Papagiannaki, “Toward the accurate identification of network applications,” in *International workshop on passive and active network measurement*. Springer, 2005, pp. 41–54.
- [18] A. Saber, B. Fergani, and M. Abbas, “Encrypted traffic classification: Combining over-and under-sampling through a pca-svm,” in *International conference on pattern analysis and intelligent systems (PAIS)*. IEEE, 2018, pp. 1–5.
- [19] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, “Robust smartphone app identification via encrypted network traffic analysis,” *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 1, pp. 63–78, 2017.
- [20] T. Van Ede, R. Bortolameotti, A. Continella, J. Ren, D. J. Dubois, M. Lindorfer, D. Choffnes, M. Van Steen, and A. Peter, “Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic,” in *Network and distributed system security symposium (NDSS)*, vol. 27, 2020, pp. 1–18.
- [21] Y. Liu, X. Wang, B. Qu, and F. Zhao, “Atvitse: A novel encrypted traffic classification method based on deep learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 9374–9389, 2024.
- [22] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, and Q. Liu, “Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification,” in *Proceedings of the ACM web conference*, 2023, pp. 2066–2075.
- [23] J. Zhou, W. Fu, H. Song, S. Yu, and Q. Xuan, “Multiview correlation-aware network traffic detection on flow hypergraph,” *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 55 922–55 935, 2025.
- [24] M. Shen, K. Ye, X. Liu, L. Zhu, J. Kang, S. Yu, Q. Li, and K. Xu, “Machine learning-powered encrypted network traffic analysis: A comprehensive survey,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 1, pp. 791–824, 2022.
- [25] R. Zhao, M. Zhan, X. Deng, Y. Wang, Y. Wang, G. Gui, and Z. Xue, “Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5420–5427.
- [26] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, “Netmamba: Efficient network traffic classification via pre-training unidirectional mamba,” in *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*. IEEE, 2024, pp. 1–11.
- [27] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *2010 IEEE symposium on security and privacy*. IEEE, 2010, pp. 305–316.

- [28] J. C. C. Chica, J. C. Imbachi, and J. F. B. Vega, "Security in sdn: A comprehensive survey," *Journal of Network and Computer Applications*, vol. 159, p. 102595, 2020.
- [29] D. Xue, R. Ramesh, A. Jain, M. Kallitsis, J. A. Halderman, J. R. Crandall, and R. Ensafi, "Openvpn is open to vpn fingerprinting," *Communications of the ACM*, vol. 68, no. 1, pp. 79–87, 2025.
- [30] B. Anderson, S. Paul, and D. McGrew, "Deciphering malware's use of tls (without decryption)," *Journal of Computer Virology and Hacking Techniques*, vol. 14, no. 3, pp. 195–211, 2018.
- [31] S. Zhu, S. Li, Z. Wang, X. Chen, Z. Qian, S. V. Krishnamurthy, K. S. Chan, and A. Swami, "You do (not) belong here: detecting dpi evasion attacks with context learning," in *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, 2020, pp. 183–197.
- [32] N. Papernot, P. McDaniel, A. Sinha, and M. P. Wellman, "Sok: Security and privacy in machine learning," in *2018 IEEE European symposium on security and privacy (EuroS&P)*. IEEE, 2018, pp. 399–414.
- [33] Y. Sharon, D. Berend, Y. Liu, A. Shabtai, and Y. Elovici, "Tantra: Timing-based adversarial network traffic reshaping attack," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3225–3237, 2022.
- [34] A. Bahramali, A. Bozorgi, and A. Houmansadr, "Realistic website fingerprinting by augmenting network traces," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1035–1049.
- [35] C. Fu, Q. Li, M. Shen, and K. Xu, "Detecting tunneled flooding traffic via deep semantic analysis of packet length patterns," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 3659–3673.
- [36] Y. Wu, "Google's neural machine translation system: Bridging the gap between human and machine translation," *arXiv preprint arXiv:1609.08144*, 2016.
- [37] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in neural information processing systems*, vol. 32, 2019.
- [38] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.
- [39] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *2017 International conference on information networking (ICOIN)*. IEEE, 2017, pp. 712–717.
- [40] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 military communications and information systems conference (MilCIS)*. IEEE, 2015, pp. 1–6.
- [41] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [42] S. Dadkhah, E. C. P. Neto, R. Ferreira, R. C. Molokwu, S. Sadeghi, and A. A. Ghorbani, "Ciciomt2024: A benchmark dataset for multi-protocol security assessment in iomt," *Internet of Things*, vol. 28, p. 101351, 2024.
- [43] G. D. Gil, A. H. Lashkari, M. Mamun, and A. A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related features," in *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP 2016)*. SciTePress Setúbal, Portugal, 2016, pp. 407–414.
- [44] A. H. Lashkari, G. D. Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of tor traffic using time based features," in *International conference on information systems security and privacy*, vol. 2. SciTePress, 2017, pp. 253–262.
- [45] N. Wickramasinghe, A. Shaghaghi, G. Tsudik, and S. Jha, "Sok: Decoding the enigma of encrypted network traffic classifiers," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 1825–1843.



Jiajun Zhou received the Ph.D degree in control theory and engineering from Zhejiang University of Technology, Hangzhou, China, in 2023. He is currently an Associate Research Fellow with the Institute of Cyberspace Security, Zhejiang University of Technology. His current research interests include graph data mining, cyberspace security and AI security.



Changhui Sun received the B.S. degree in network engineering from Wenzhou University, Wenzhou, China, in 2024. He is currently pursuing the M.S. degree in computer science and technology at Zhejiang University of Technology, Hangzhou, China. His current research interests include cyberspace security and network traffic detection.



Wentao Fu received the B.S. degree in Automation from Xi'an University of Posts and Telecommunications, Xi'an, China, in 2023. He is currently pursuing his master's degree at the Institute of Cyberspace Security, Zhejiang University of Technology, China. His current research interests include cyberspace security and network intrusion detection.



Meng Shen (Member, IEEE) received the B.Eng. degree in computer science from Shandong University, Jinan, China, in 2009, and the Ph.D. degree in computer science from Tsinghua University, Beijing, China, in 2014. He is currently a Professor at Beijing Institute of Technology, Beijing. He has authored more than 50 papers in top-level journals and conferences, such as USENIX Security, ACM CCS, IEEE Journal on Selected Areas in Communications, and IEEE Transactions on Information Forensics and Security. His research interests include data privacy and security, blockchain applications, and encrypted traffic classification. He received the Best Paper Runner-Up Award from IEEE IPCCC 2014 and IEEE/ACM IWQoS 2020. He was selected by the Beijing Nova Program 2020 and the winner of the ACM SIGCOMM China Rising Star Award 2019. He has guest edited Special Issues on Emerging Technologies for Data Security and Privacy in IEEE Network and IEEE Internet of Things Journal.



Shanqing Yu received the M.S. degree from the School of Computer Engineering and Science, Shanghai University, China, in 2008 and received the M.S. degree from the Graduate School of Information, Production and Systems, Waseda University, Japan, in 2008, and the Ph.D. degree, in 2011, respectively. She is currently a Lecturer at the Institute of Cyberspace Security and the College of Information Engineering, Zhejiang University of Technology, Hangzhou, China. Her research interests cover intelligent computation and data mining.



Qi Xuan (M'18) received the BS and PhD degrees in control theory and engineering from Zhejiang University, Hangzhou, China, in 2003 and 2008, respectively. He was a Post-Doctoral Researcher with the Department of Information Science and Electronic Engineering, Zhejiang University, from 2008 to 2010, respectively, and a Research Assistant with the Department of Electronic Engineering, City University of Hong Kong, Hong Kong, in 2010 and 2017. From 2012 to 2014, he was a Post-Doctoral Fellow with the Department of Computer Science, University of California at Davis, CA, USA. He is a senior member of the IEEE and is currently a Professor with the Institute of Cyberspace Security, College of Information Engineering, Zhejiang University of Technology, Hangzhou, China. His current research interests include network science, graph data mining, cyberspace security, machine learning, and computer vision.

A. Pseudocode of Model Training

Algorithm 1 outlines the unified training pipeline of *Traffic-MoE*, detailing the progression from self-supervised pre-training to task-specific fine-tuning.

Algorithm 1 Pretraining and Fine-tuning of *Traffic-MoE*.

```

1: Input: batch of flow token sequences  $\{X^{(b)}\}_{b=1}^B$ , training
   mode  $\star \in \{\text{PRETRAIN}, \text{FINETUNE}\}$ , labels  $\{y_b\}_{b=1}^B$  if
   FINETUNE;
2: hyperparameters: transformer depth  $L$ , number of ex-
   perts  $N$ , top- $k$  routing  $k$ , auxiliary weight  $\lambda_{\text{aux}}$ ;
3: Output: updated model and task-specific head;
   /* Backbone: causal transformer with sparse MoE */
4: for  $b = 1$  to  $B$  do
5:   Initialize hidden states for the input token sequences:
      $H^{(0,b)} \leftarrow X^{(b)}$ ;
6:   for  $l = 1$  to  $L$  do
7:     Compute causal self-attention  $O_{\text{attn}}^{(l,b)}$  via Eq.(1)~(2);
8:     Residual update:  $\widetilde{H}^{(l,b)} \leftarrow H^{(l-1,b)} + O_{\text{attn}}^{(l,b)}$ ;
9:     Apply sparse MoE to compute  $H^{(l,b)}$  via Eq.(4)~(8);
10:   end for
11: end for
12: Compute MoE load-balancing loss  $\mathcal{L}_{\text{aux}}$  via Eq.(9);
   /* Pretraining */
13: if  $\star$  is PRETRAIN then
14:   Compute autoregressive loss  $\mathcal{L}_{\text{NP}}$  via Eq.(11);
15:   Combine objectives:  $\mathcal{L}_{\text{pretrain}} \leftarrow \mathcal{L}_{\text{NP}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}$ ;
16:   Update backbone parameters by minimizing  $\mathcal{L}_{\text{pretrain}}$ ;
   /* Fine-tuning */
17: else
18:   For each flow  $X^{(b)}$ , perform mean pooling over final-
     layer tokens and classify to obtain  $p_{\theta}(y_b | X^{(b)})$ ;
19:   Compute classification loss  $\mathcal{L}_{\text{TC}}$  via Eq.(14);
20:   Combine objectives:  $\mathcal{L}_{\text{finetune}} \leftarrow \mathcal{L}_{\text{TC}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}}$ ;
21:   Update backbone and prediction head by minimizing
      $\mathcal{L}_{\text{finetune}}$  with layer-wise learning-rate decay;
22: end if

```

B. Fine-tuning Datasets

To systematically evaluate the transfer learning capabilities of model across diverse dimensions, we employ five authoritative public datasets during the fine-tuning phase.

- **CICIoT2023** [41]: Collected from a complex network environment comprising 105 real IoT devices, this dataset contains benign background traffic and 33 attack behaviors. These attacks are grouped into 7 macro-categories, including DDoS, Brute Force, Web attacks, and Mirai botnets.
- **CICIoMT2024** [42]: Derived from a testbed of 40 real and simulated IoMT devices, this dataset encompasses 18 targeted attacks (grouped into 5 categories) exploiting standard healthcare protocols such as Wi-Fi and MQTT.
- **ISCXVPN2016** [43]: This dataset comprises both regular and VPN-encapsulated session traffic. It covers 14 fine-

grained traffic categories (e.g., VoIP, P2P, Email) generated by diverse applications like Facebook, Skype, and Chrome.

- **ISCXTor2016** [44]: This dataset includes regular and Tor-obfuscated session traffic. It spans 8 service categories involving applications such as YouTube, Firefox, and Skype.
- **CipherSpectrum** [45]: This dataset features 2024-collected TLS 1.3 encrypted traffic originating from 40 web domains. All sessions are strictly encrypted using the three mainstream AEAD cipher suites mandated by TLS 1.3 protocol.

C. Details of Comparison Methods

- **AppScanner** [19]: A classic statistical fingerprinting method. It extracts 54 statistical features from encrypted traffic and employs a Random Forest classifier to identify applications without payload inspection.
- **FlowPrint** [20]: A semi-supervised approach designed for open-world scenarios. It mines temporal correlations within traffic to construct a dynamic fingerprint library, enabling the identification of previously unseen applications.
- **FS-Net** [10]: An end-to-end deep learning architecture tailored for sequence modeling. It utilizes a Bidirectional Gated Recurrent Unit in an encoder-decoder structure to learn latent representations directly from raw packet length sequences, bypassing manual feature engineering.
- **ET-BERT** [11]: A pioneering method adapting the BERT architecture to encrypted traffic domain. It introduces two pre-training objectives (Masked Burst Model and Same-origin Burst Prediction) to learn contextual dependencies from massive unlabeled flow data.
- **NetGPT** [12]: The first framework applying Generative Pre-trained Transformers (GPT) to the traffic domain. It treats network flows as hexadecimal byte sequences and unifies traffic understanding and generation tasks, adapting to downstream tasks via prompt tuning.
- **TrafficFormer** [13]: A recent advancement building upon ET-BERT. It introduces a Same-Origin-Direction-Flow multi-task objective to capture fine-grained packet directionality and sequence relationships more effectively.
- **YaTC** [25]: A Transformer-based traffic classification model utilizing Masked Autoencoder and multi-level flow representations. It employs dual-level attention mechanisms to extract features, achieving efficient encrypted traffic classification via the pre-training paradigm.
- **NetMamba** [26]: An efficient pre-trained traffic classification model using unidirectional Mamba. Combined with optimized traffic representation schemes, it significantly accelerates inference while maintaining robust accuracy.

Note on Implementation: To ensure a fair comparison of *model capacity* rather than data engineering tricks, we standardize the experimental setting. We exclude method-specific data augmentation strategies used in the original papers and train all models on identical raw datasets.

D. Fine-grained Categories to Merged Classes

Table IV details the mapping from fine-grained categories to merged classes for *CICIoMT2024* and *CICIoT2023*.

TABLE IV: Attack category mapping scheme for the CIIoMT2024 and CIIoT2023 Datasets.

Merged Categories		Fine-grained Categories
CIIoMT2024	Benign	BenignTraffic
	DDoS	MQTT-DDoS-Connect_Flood, MQTT-DDoS-Publish_Flood, TCP_IP-DDoS-ICMP, TCP_IP-DDoS-SYN, TCP_IP-DDoS-TCP, TCP_IP-DDoS-UDP
	DoS	MQTT-DoS-Connect_Flood, MQTT-DoS-Publish_Flood, TCP_IP-DoS-ICMP, TCP_IP-DoS-SYN, TCP_IP-DoS-TCP, TCP_IP-DoS-UDP
	Malformed	MQTT-Malformed_Data
	Recon	Recon-OS_Scan, Recon-Ping_Sweep, Recon-Port_Scan, Recon-VulScan
	Spoofing	ARP_Spoofing
Merged Categories		Fine-grained Categories
CIIoT2023	Benign	BenignTraffic
	BruteForce	DictionaryBruteForce
	DDoS	DDoS-ACK-Fragmentation, DDoS-HTTP-Flood, DDoS-ICMP-Flood, DDoS-ICMP-Fragmentation, DDoS-PSHACK-Flood, DDoS-RSTFINFlood, DDoS-SYN-Flood, DDoS-SlowLoris, DDoS-SynonymousIP-Flood, DDoS-TCP-Flood, DDoS-UDP-Flood, DDoS-UDP-Fragmentation
		DoS-HTTP-Flood, DoS-SYN-Flood, DoS-TCP-Flood, DoS-UDP-Flood
		Mirai-greeth-flood, Mirai-greip-flood, Mirai-upplain
		Recon-HostDiscovery, Recon-OSScan, Recon-PingSweep,
	Recon	Recon-PortScan, VulnerabilityScan
	Spoofing	DNS-Spoofing, MITM-Arpspoofing
	WebAttack	Backdoor/Malware, BrowserHijacking, CommandInjection, SqlInjection, Uploading-Attack, XSS

E. Efficacy and Robustness Analysis

To investigate the performance boundaries of *Traffic-MoE* in real-world security scenarios, we conducts an in-depth analysis of fine-grained results, as shown in Table V and Figure 9.

1) *Intrusion Detection: Minimizing Security Risks:* In the context of network defense, the metrics for the *Benign* category serve as global indicators for system reliability: the **Benign FPR** represents the *Overall Attack Miss Rate* (attacks misclassified as benign), while the **Benign FNR** represents the *False Alarm Rate* (benign traffic misclassified as attacks). *Traffic-MoE* demonstrates exceptional capability in preventing defense penetration, achieving a negligible Attack Miss Rate (Benign FPR) of 0.11% on the sensitive *CIIoMT2024* dataset and a robust 3.14% on the massive *CIIoT2023* dataset. This implies that the vast majority of threats are successfully intercepted. Conversely, the False Alarm Rate (Benign FNR) is observed at 11.18% in IoT scenarios. This reflects a “Fail-Secure” design philosophy: given the intrinsic behavioral overlap between aggressive benign polling and malicious reconnaissance in IoT networks, the model prioritizes flagging ambiguous traffic for inspection rather than risking a breach. While volumetric attacks are detected with high precision, stealthier categories like *Spoofing* exhibit performance dips, suggesting that low-signature attacks remain a challenge for generalized representations.

2) *Tor Anonymity: Countering Traffic Shaping:* In adversarial anonymity scenarios, *Traffic-MoE* demonstrates a unique ability to counter traffic shaping mechanisms. These operations of fixed-size cell padding and randomized delays used by Tor protocol effectively distort the statistical feature space, leading to significant performance degradation for latency-sensitive services like *Audio* and transactional services like *Email* (as shown in Figure 9). However, *Traffic-MoE* exhibits resilience in interactive protocols such as *P2P* and *VoIP*, where detection accuracy remains high. This performance dichotomy indicates that the MoE architecture successfully shifts attention from

TABLE V: Detection performance of *Traffic-MoE* on different traffic categories (merged) in CIIoMT2024 and CIIoT2023

	Merged Category	Num.#Test	Recall	FNR	FPR
CIIoMT2024	Benign	130	0.9462	0.0538	0.0011
	DDoS	4193	0.9955	0.0045	0.0046
	DoS	4075	0.9931	0.0069	0.0029
	Malformed	45	0.9333	0.0667	0.0001
	Recon	2229	0.9928	0.0072	0.0013
	Spoofing	67	0.7015	0.2985	0.0019
	Merged Category	Num.#Test	Recall	FNR	FPR
CIIoT2023	Benign	3534	0.8882	0.1118	0.0314
	BruteForce	265	0.7925	0.2075	0.0008
	DDoS	10775	0.9426	0.0574	0.0330
	DoS	4000	0.9785	0.0215	0.0021
	Mirai/Botnet	663	0.6591	0.3409	0.0084
	Recon	3754	0.8426	0.1574	0.0289
	Spoofing	2000	0.8250	0.1750	0.0125
	WebAttack	527	0.6831	0.3169	0.0045

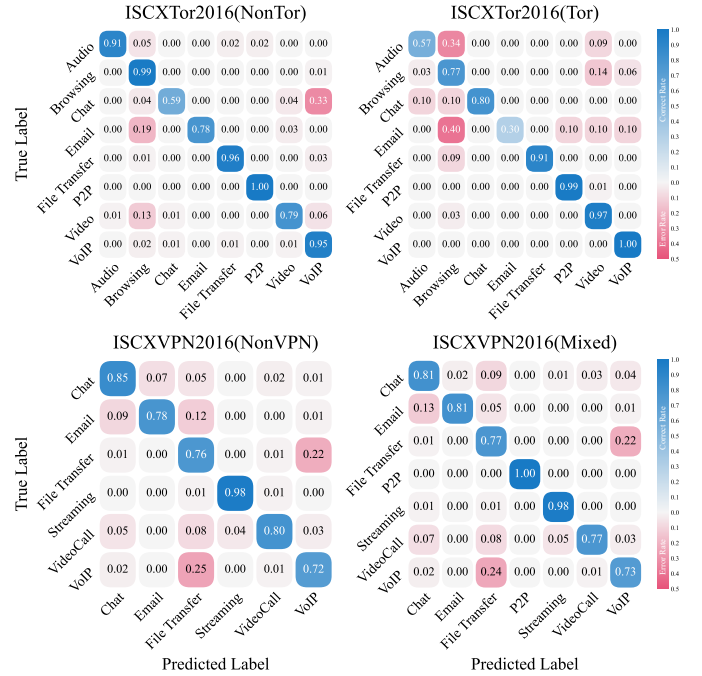


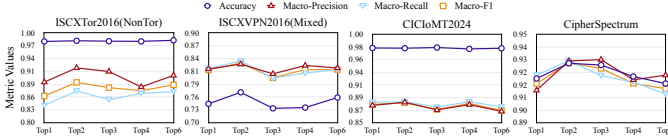
Fig. 9: Performance confusion matrices of *Traffic-MoE* across different traffic classification tasks.

eroded packet-length statistics to *sequence-level behavioral fingerprints* (e.g., handshake orders and interaction frequencies), effectively learning a “Protocol Grammar” that persists despite cryptographic obfuscation.

3) *VPN Tunneling: Penetrating Encapsulation:* Similarly, under VPN tunneling conditions (*ISCVPN2016*), *Traffic-MoE* demonstrates strong resilience to protocol encapsulation. The confusion matrix reveals that misclassifications (e.g., between VoIP and File Transfer) are consistent across Non-VPN and Mixed scenarios, stemming from intrinsic feature similarity rather than tunnel noise. Crucially, for complex classes like Streaming and P2P, the model maintains near-perfect identification. This suggests that the self-attention mechanism effectively penetrates the outer tunnel headers to capture long-

TABLE VI: More ablation study results across different traffic detection tasks (Appendix)

Ablation Variant	Task 1: ISCXTor2016(Tor)				Task 2: ISCXVPN2016(NonVPN)				Task 3: CICIoT2023			
	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1	ACC	M-PR	M-RC	M-F1
Traffic-MoE(Ours)	0.9089	0.8942	0.7879	0.8072	0.7613	0.7866	0.8158	0.8005	0.8588	0.8007	0.7701	0.7824
MoE→Dense(Total)	0.9038	0.8656	0.7638	0.7767	0.7178	0.7570	0.7900	0.7713	0.8594	0.8090	0.7765	0.7894
MoE→Dense(Active)	0.8962	0.8676	0.7774	0.7930	0.7258	0.7632	0.7853	0.7696	0.8521	0.7878	0.7607	0.7704
w/o Shared Expert	0.8557	0.7433	0.6846	0.6855	0.6268	0.6711	0.6470	0.6440	0.7627	0.7208	0.6798	0.6820
w/o PT	0.8304	0.7071	0.6392	0.6311	0.6062	0.6327	0.6095	0.6153	0.7833	0.6757	0.6584	0.6640
Dense(Total) w/o PT	0.8506	0.7056	0.6541	0.6564	0.6074	0.6334	0.6533	0.6389	0.7867	0.7054	0.6586	0.6741
w/o $\mathcal{L}_{aux}$	0.8962	0.8734	0.7683	0.7913	0.7347	0.7803	0.7949	0.7829	0.8535	0.8002	0.7630	0.7809
w/o Traffic2Token	0.8835	0.8273	0.7460	0.7645	0.6537	0.7259	0.7397	0.7312	0.8365	0.7789	0.7358	0.7523
Traffic-MoE(Ours)	0.9089	0.8942	0.7879	0.8072	0.7613	0.7866	0.8158	0.8005	0.8588	0.8007	0.7701	0.7824
w/ Header Only	0.8759	0.7721	0.7106	0.6996	0.7308	0.7665	0.7846	0.7715	0.8395	0.7679	0.7211	0.7358
w/ Payload Only	0.6227	0.6397	0.5566	0.5696	0.4967	0.6236	0.5445	0.5472	0.3965	0.5782	0.3314	0.3710
w/o Payload-Data	0.9068	0.8944	0.7798	0.8096	0.7610	0.7856	0.8097	0.7931	0.8583	0.8045	0.7713	0.7871
w/o Payload-Negotiation	0.7435	0.7095	0.6073	0.6242	0.6217	0.6758	0.6381	0.6087	0.7022	0.6982	0.6163	0.6389
w/o Header-Length	0.7975	0.8618	0.5973	0.6004	0.6743	0.6383	0.6677	0.6478	0.5235	0.6155	0.5178	0.4636
w/o Header-Direction	0.8937	0.8912	0.7713	0.7961	0.7349	0.7449	0.7717	0.7572	0.6747	0.7095	0.6492	0.6199
w/o Header-Flags	0.8684	0.8910	0.6870	0.7204	0.7015	0.7371	0.7263	0.7311	0.6504	0.7152	0.6244	0.6219
w/o Header-Proto	0.9013	0.8937	0.7702	0.7844	0.7513	0.7768	0.7945	0.7848	0.7928	0.7157	0.6794	0.6781
w/o Header-Time	0.7595	0.6618	0.5808	0.5938	0.5054	0.6344	0.5943	0.5578	0.5521	0.6436	0.4282	0.4407

Fig. 10: Impact of Top- k parameters on model performance.

range flow dependencies, achieving semantic decoupling of the payload behavior from the transport encapsulation.

F. Extended Ablation Analysis

Table VI provides additional ablation results across three traffic detection tasks.

G. Hyperparameter Analysis of the Top- k Routing Strategy

To assess the trade-off between representation capacity and computational sparsity, we evaluate varying the number of active experts, $k \in \{1, 2, 3, 4, 6\}$ (Figure 10). Results show that across most scenarios, performance peaks at $k = 2$, achieving Pareto optimality between accuracy and inference efficiency.

Specifically, increasing k from 1 to 2 yields significant gains, primarily attributed to collaborative synergy between “primary-auxiliary” experts in feature extraction. However, as k further increases, the model exhibits performance fluctuation and degradation across all scenarios (e.g., *ISCVN2016*). This performance decay reveals the core mechanism of the MoE architecture: its advantages stem from sparse activation and specialized isolation rather than global computation. Under the design constraint of a constant active parameter count, forcing the activation of more experts leads to a severe dilution of parameter capacity assigned to each individual expert, thereby weakening their representational power. Crucially, a larger k forces heterogeneous traffic features to share the same expert subsets, breaking the established state of specialized isolation. This triggers significant feature interference and blurs the classification boundaries. In extreme

cases, an excessive increase in k causes the MoE architecture to degenerate into a structure resembling traditional dense networks, significantly undermining the model’s efficacy in processing complex, heterogeneous network traffic.

H. Parameter Analysis on Traffic Sequence

To explore the optimal sequence truncation strategy and balance representational capacity with computational overhead, we conducted an ablation study on the payload truncation length (J) and the number of truncated packets (K).

Impact of Payload Truncation Length (J): As shown in Figure 11, the model’s performance exhibits an overall upward trend as J increases from 10 to 40 bytes, after which it plateaus. This phenomenon validates the inherent “header semantic concentration” characteristic of network traffic: critical discriminative fingerprints (e.g., TLS record headers and cipher suite identifiers) are highly clustered at the beginning of the payload. Regions beyond this threshold mostly consist of high-entropy encrypted data or random padding, which lack discriminative power and act as noise that disrupts the attention mechanism’s focus. Therefore, setting $J = 40$ precisely extracts high-density discriminative semantics while effectively bounding the token sequence length, drastically reducing FLOPs during inference.

Impact of Packet Count (K): As Figure 12 illustrates, the majority of task scenarios exhibit a clear pattern of diminishing marginal returns. Performance significantly improves as K increases up to 10; however, for $K > 10$, it generally plateaus or fluctuates. This occurs because the most discriminative “protocol syntax” (e.g., TCP/TLS handshakes) manifests entirely during the initial connection establishment phase. Once entering the bulk data transfer phase, traffic behavior becomes highly homogenized, offering negligible incremental information. Thus, for most tasks, $K = 10$ is the optimal solution to fully capture session logic while preventing the quadratic growth of computational complexity.

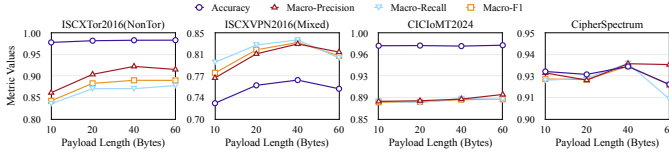


Fig. 11: Impact of payload length on model performance.

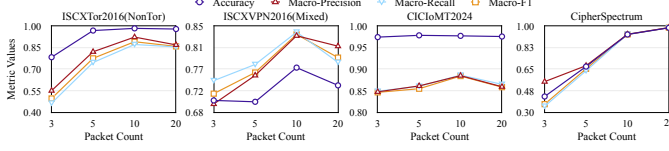


Fig. 12: Impact of packet count on model performance.

Notably, in the modern pure TLS 1.3 encrypted scenario (*CipherSpectrum*), expanding the observation window yields continuous gains. Although the model already demonstrates excellent discriminative capability at $K = 10$, increasing K to 20 achieves further breakthroughs. This reflects a TLS 1.3 representational shift: because comprehensive encryption drastically compresses plaintext handshake phase, more discriminative features are implicitly encoded within mid-to-late interactions (e.g., application-level packet length evolution and temporal burst). Consequently, longer observation window effectively unlocks these deep behavioral features, supplying abundant structural cues to sharpen the decision boundary.

I. Detailed Evaluation under Distribution Shifts

- **Time-shift** simulates concept drift. For fine-grained traffic classification tasks (e.g., distinguish DDoS sub-types), we strictly sort all session flows of each category by timestamp. We designate the first 40% of the time span as the training domain and the last 40% as the testing domain, discarding the middle 20% to establish a temporal buffer. This “past predicting future” setting eliminates temporal data leakage and ensures a significant temporal distributional gap.
- **Proportion-shift** evaluates the model’s adaptability to intra-class distribution shifts. For coarse-grained traffic classification tasks, we artificially adjust the ratio of dominant to minor components in each coarse-grained category (e.g., DDoS, WebAttack): the training set is sampled with a 4:1 ratio, whereas the testing set adopts a completely inverse 1:4 ratio. This extreme distribution inversion rigorously tests the model’s capacity to capture long-tail features.
- **Compose-shift** simulates unseen attack sub-variants. In coarse-grained traffic classification tasks, we randomly mask 50% of fine-grained sub-class traffic per category during training, while the testing set includes all sub-classes, thereby simulating the real-world challenge of “missing information” due to emerging attack variants.

1) *Resilience to Concept Drift (Time-shift)*: The severe performance degradation of dense baselines like ET-BERT under the Time-shift scenario exposes their structural reliance on *temporal shortcuts*. These shortcuts include transient statistical artifacts tied to the training timeframe, which inevitably become invalid as the network context evolves. Conversely,

the sparse expert specialization of *Traffic-MoE* naturally resists this overfitting. By selectively activating experts grounded in pre-trained protocol knowledge, the architecture inherently filters out temporal noise and isolates invariant attack behaviors, maintaining high robustness against concept drift.

2) *Mitigation of Gradient Starvation (Proportion-shift)*: The competitive performance under intra-class distribution shifts shows our MoE architecture effectively mitigates the gradient starvation of minority classes typically encountered in dense networks. The routing mechanism acts as a divide-and-conquer strategy, assigning long-tail minor components to specific experts. This prevents the gradient updates of these minority variants from being diluted by dominant classes, ensuring high recall even under inverted test distributions.

3) *Adaptation to Unseen Variants (Compose-shift)*: In scenarios containing unseen sub-classes, the consistently high recall of *Traffic-MoE* further validates its capacity for behavioral abstraction. Instead of overfitting to tool-specific payloads, the model captures generic malicious patterns (e.g., structural characteristics of flooding attack). This ability to generalize rather than simply memorize signatures is critical for identifying emerging threats, particularly in operational environments where evasions carry severe security costs.

By effectively managing the inherent non-stationarity of real-world traffic, *Traffic-MoE* avoids catastrophic performance drops common in dense models. Its superior generalization enables sustained adaptation to evolving network environments, significantly reducing the frequency and computational overhead of model retraining in long-term deployments.

J. Empirical Justification for Dense Attention Mechanism

To empirically justify the design choice of employing dense attention in *Traffic-MoE* for capturing universal traffic syntax, we visualize the attention distributions of representative traffic categories across four detection scenarios (see Figure 13). We average the multi-head attention matrices at specific model layers over the entire test set and apply padding masks to ensure the computation focuses on valid traffic tokens. For maximum information density, the visualization focuses on the first half of flow sequences and utilizes the “Attention-to-Baseline Ratio” as a quantitative metric, where $1\times$ represents the uniform attention baseline.

Capturing Universal Protocol Syntax. Despite significant semantic divergence among different traffic categories, their attention patterns exhibit striking structural consistency at the same model depth. This indicates that the dense attention module effectively functions as a *universal grammars parser*. Since the vast majority of network traffic (whether benign business flows or malicious attacks) strictly adheres to foundational TCP/IP state machines and structural delimiters, the attention mechanism successfully extracts cross-class shared representations rather than overfitting to class-specific signatures.

Interpretation of Attention Patterns. The observed attention distributions align with the physical properties of network protocols. First, we observe a pronounced *header focus*. Driven by causal mask dilution and the high information entropy inherent in initial flow negotiations and metadata exchanges,

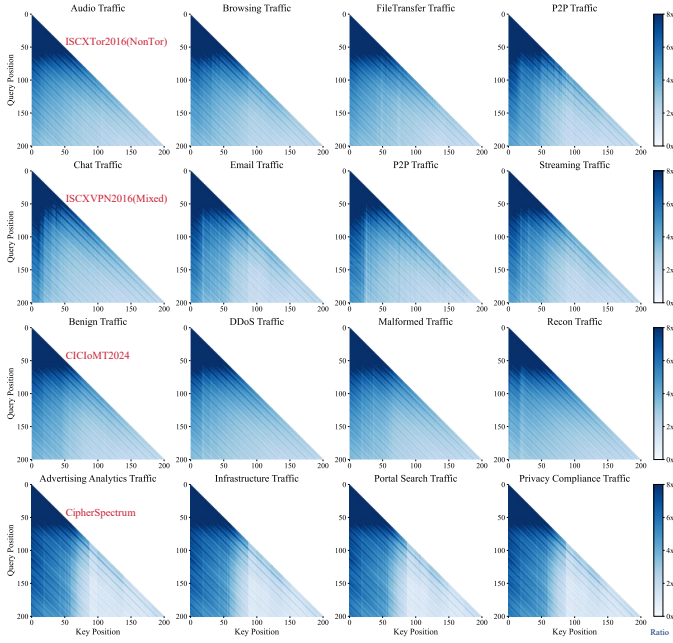


Fig. 13: Attention distributions of traffic categories across detection tasks.

attention concentrates at the flow sequence start. Local attention peaks in this region reach up to $8\times$ the baseline, quantitatively validating the backbone’s precision in parsing foundational syntax. Second, the distinct diagonal stripes in the heatmaps reveal the sequential parsing of local byte-level transitions. This localized context extraction naturally complements the global long-range dependencies captured by the header focus, collectively enabling the model to construct comprehensive hierarchical representations.

Strategic Architectural Choice. Since network traffic syntax remains invariant across distinct categories, imposing artificial sparsity constraints on the attention mechanism would introduce unnecessary inductive bias, resulting in a suboptimal architecture. Our empirical findings substantiate that the functionally decoupled design (*dense attention for universal syntax extraction, paired with sparse MoE for specialized semantic modeling*) achieves optimal representational capacity with minimal computational redundancy. This architectural synergy fundamentally explains how *Traffic-MoE* sustains robust generalization while maintaining highly efficient inference.